

CORE JAVA NOTES

Roadmap: [Core Java Roadmap](#)

1. Java Basics

1.1 Data Types

- 1.1.1 byte
- 1.1.2 short
- 1.1.3 int
- 1.1.4 long
- 1.1.5 float
- 1.1.6 double
- 1.1.7 char
- 1.1.8 boolean

1.2 Literals

- 1.2.1 Integer Literals
- 1.2.2 Floating Point Literals
- 1.2.3 Character Literals
- 1.2.4 String Literals
- 1.2.5 Boolean Literals
- 1.2.6 Null Literal

1.3 Operators

- 1.3.1 Arithmetic Operators
- 1.3.2 Unary Operators
- 1.3.3 Relational / Comparison Operators
- 1.3.4 Logical Operators
- 1.3.5 Bitwise Operators
- 1.3.6 Assignment Operators
- 1.3.7 Ternary Operator
- 1.3.8 Instanceof Operator

1.4 Type Conversion and Type Promotion

- 1.4.1 Implicit Type Conversion
- 1.4.2 Explicit Type Conversion
- 1.4.3 Type Promotion

2. Control Statements

2.1 If Statement

2.2 If-Else Statement

2.3 If-Else Ladder

2.4 Switch Case

3. Loops

- 3.1 Do-While Loop
- 3.2 While Loop
- 3.3 For Loop
- 3.4 Enhanced For Loop

4. JDK, JRE, and JVM

- 4.1 JDK
- 4.2 JRE
- 4.3 JVM

5. Basic Data Structures

- 5.1 Array
 - 5.1.1 1-D Array
 - 5.1.2 2-D Array
 - 5.1.3 Multi-Dimensional Array
- 5.2 Strings

6. Memory Management

- 6.1 Stack Memory
- 6.2 Heap Memory
- 6.3 Garbage Collection

7. Wrapper Classes

- 7.1 Wrapper Class Concept
- 7.2 Autoboxing and Unboxing

8. Object-Oriented Programming

- 8.1 Class and Object
- 8.2 Constructor
- 8.3 Inheritance
- 8.4 Encapsulation
- 8.5 Polymorphism
 - 8.5.1 Compile-Time Polymorphism
 - 8.5.1.1 Method Overloading
 - 8.5.2 Run-Time Polymorphism
 - 8.5.2.1 Method Overriding

8.5.2.2 Dynamic Method Dispatch
8.6 Abstraction

9. Abstract Classes

9.1 Abstract Class
9.2 Inner Class
9.3 Anonymous Inner Class

10. Interfaces

10.1 Normal Interface
10.2 Functional Interface
10.3 Marker Interface

11. Enums

11.1 Enum Basics
11.2 Enum Use Cases

12. Functional Programming

12.1 Lambda Expressions

13. Exception Handling

13.1 try
13.2 catch
13.3 throw
13.4 throws
13.5 finally
13.6 try-with-resources

14. Input Taking Methods

14.1 Scanner
14.2 BufferedReader

15. Multithreading

15.1 Thread Basics
15.2 Thread Priority and Sleep
15.3 Runnable vs Thread
15.4 Thread States
15.5 Race Condition

16. Collection Framework

16.1 Interfaces

16.1.1 Collection

16.1.2 List

16.1.2.1 Internal Working

16.1.2.2 ArrayList

16.1.2.3 LinkedList

16.1.3 Set

16.1.3.1 Set

16.1.3.2 TreeSet

16.1.4 Queue

16.1.5 Map

16.1.5.1 HashMap

16.1.5.2 TreeMap

16.2 Classes

16.2.1 ArrayList

16.2.2 LinkedList

16.2.3 HashSet

16.2.4 TreeSet

16.3 Comparator vs Comparable

17. Java 8 Features

17.1 Stream API

17.1.1 map

17.1.2 reduce

17.1.3 filter

17.2 Method Reference

17.3 Optional Class

17.4 Date-Time API

Class 01: Getting Started with Java

1. A Short History of Java

- **Java** is a high-level, object-oriented programming language.
 - Originally developed by **James Gosling** at **Sun Microsystems** in **1995**.
 - Java was later acquired by **Oracle Corporation** in **2010**.
 - Designed with the concept of:
 - ◆ *"Write Once, Run Anywhere"* (WORA)
- This means you can write one time and run this java code on any machine you want.
-

2. Why Java is Powerful & Popular

- Platform Independent (thanks to JVM)
 - Object-Oriented
 - Secure & Robust
 - Multithreaded
 - Huge ecosystem: Spring Boot, Android, Big Data, etc.
 - Massive community support and libraries
-

3. JDK vs JRE vs JVM

Component	Description
JDK (Java Development Kit)	A full development kit. Contains JRE + development tools like <code>javac</code> , <code>javadoc</code> , etc.
JRE (Java Runtime Environment)	Needed to run Java programs. Contains JVM + libraries and classes.
JVM (Java Virtual Machine)	Converts bytecode to machine code. Platform-dependent. Core of Java's portability.

Think of it like:

- **Hardware** → Installed OS → Inside OS: JDK → Inside JDK: JRE → Inside JRE: JVM
-

4. How JVM Executes Java Code

1. You write a `.java` file
 2. You compile it using `javac` → Produces `.class` file (Bytecode)
 3. Bytecode is platform-independent
 4. JVM loads the `.class` file
 5. JVM **scans all lines of code**, searches for the `main()` method
 6. **Execution starts from `main()`**
-

5. Your First Java Program

```
// File: HelloWorld.java
public class HelloWorld {
    public static void main(String[] args) {
        System.out.println("Hello, world!");
    }
}
```

Steps to Run:

```
javac HelloWorld.java    // Compiles to HelloWorld.class
java HelloWorld          // Runs the program
```

6. WORA: Write Once, Run Anywhere

- Java compiles to **bytecode**, not machine code.
- Bytecode is interpreted by the **JVM** on any platform (Windows, Mac, Linux).
- Hence, you can run the **same .class file on any OS** that has a JVM.

Important Note:

Java is platform-independent, but **JVM is platform-dependent**

→ You need the **correct JVM** for your OS (Windows JVM for Windows, etc.)

Variables:

Variable in Java is a container that holds data which can be changed during the execution of a program.

Syntax

```
dataType variableName = value;
```

Type	Where Declared	Scope	Example
Local Variable	Inside a method or block	Only within that method/block	<code>int age = 25;</code>

We will see more types of variables when we learn about classes.

```
public class VariableExample {  
    public void showDetails() {  
        // Local variable  
        int age = 25;  
        System.out.println("Age: " + age);  
    }  
}
```

Java Variable Naming Rules

1. Must begin with a letter, _, or \$.
2. Can't use keywords (e.g., int, class) as names.
3. Case-sensitive (num ≠ Num).
4. Use camelCase for naming (e.g., studentAge, totalMarks).

Tips:

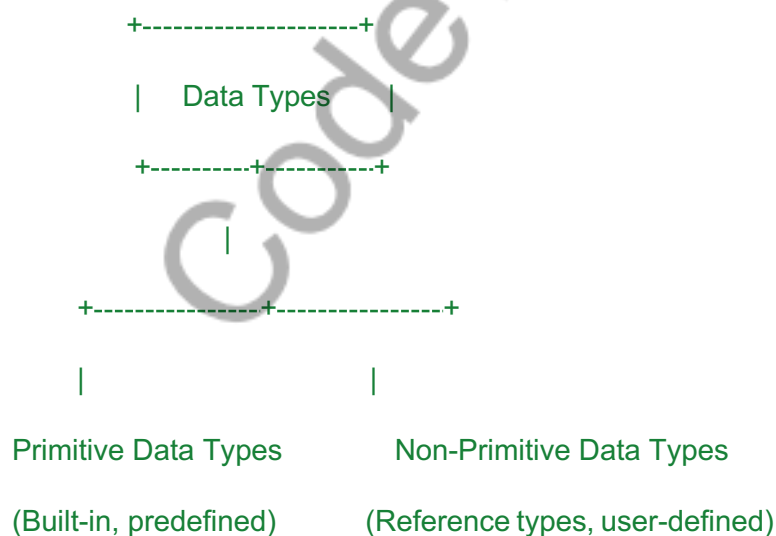
Initialize local variables before use (Java will not auto-initialize them).

Data Types

What is a Data Type?

A **data type** specifies the type of data that a variable can store. Java is a **strongly typed language**, meaning every variable must be declared with a type.

Java Data Types are Divided Into Two Main Categories:



1. Primitive Data Types (8 types)

Java has **8 primitive data types**. These are basic types defined by the language.

Category	Type	Size	Default Value	Example
Integer	byte	1 byte	0	byte a = 10;
	short	2 bytes	0	short s = 1000;
	int	4 bytes	0	int x = 50000;
	long	8 bytes	0L	long l = 123456L;
Floating	float	4 bytes	0.0f	float f = 3.14f;
	double	8 bytes	0.0d	double d = 99.99;
Character	char	2 bytes	'\u0000'	char c = 'A';
Boolean	boolean	1 bit	false	boolean b = true;

◆ Integer Types

Used for whole numbers:

```
byte b = 100;
```

```
short s = 10000;
```

```
int i = 100000;
```

```
long l = 100000L;
```

◆ Floating Point Types

Used for decimal numbers:

```
float pi = 3.14f;
```

```
double price = 99.99;
```

◆ Character Type

```
char initial = 'J';
```

◆ Boolean Type

```
boolean isJavaFun = true;
```

2. Non-Primitive Data Types (Reference Types)

Non-primitive data types **refer to objects**, not actual values. We will see them in upcoming classes.

Examples:

- **Strings**
- **Arrays**
- **Classes**
- **Interfaces**
- **Enums**

Literals

What is Literal in Java?

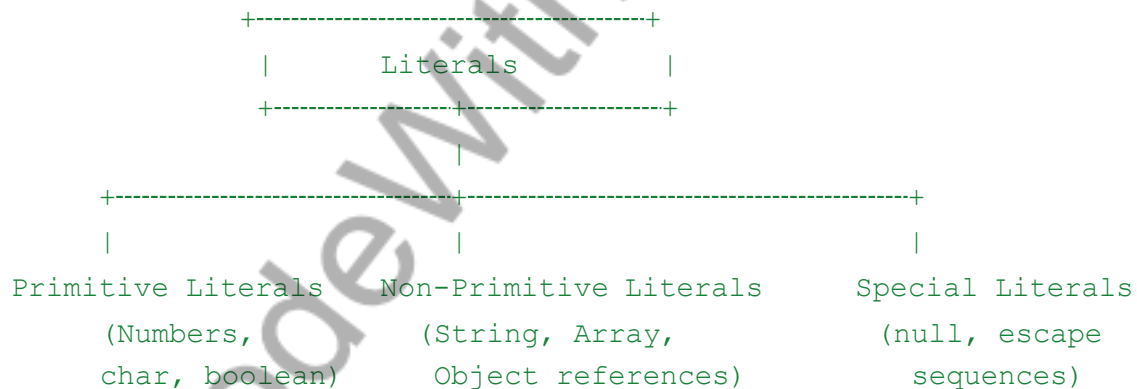
A **literal** is a fixed value assigned directly to a variable in the code.

It represents constant values that are **used directly in the code without computation**. Think of it as the **actual value** you assign:

```
int age = 25; // 25 is a literal
```

Types of Literals in Java

Java supports several types of literals depending on the **data type**.



1. Integer Literals

Used with types: `byte`, `short`, `int`, `long`

➤ Decimal (Base 10)

```
int a = 100;
```

➤ **Octal (Base 8 – starts with 0)**

```
int b = 075;    // 61 in decimal
```

➤ **Hexadecimal (Base 16 – starts with 0x or 0X)**

```
int c = 0xFF;   // 255 in decimal
```

➤ **Binary (Base 2 – starts with 0b or 0B)**

```
int d = 0b1010; // 10 in decimal
```

➤ **Long Literal (Ends with `L` or `l`)**

```
long bigNumber = 10000000000L;
```

✓ 2. Floating-Point Literals

Used with: `float`, `double`

➤ **Double (default for decimals)**

```
double price = 99.99;
```

➤ **Float (requires `f` or `F`)**

```
float pi = 3.14f;
```

➤ **Scientific Notation (exponential form)**

```
double e = 1.23e2; // 123.0
```

3. Character Literals

Single characters inside **single quotes**

```
char letter = 'A';  
char digit = '7';
```

➤ **Unicode characters**

```
char uni = '\u0041'; // 'A'
```


Escape Sequences

Escape	Meaning
<code>\n</code>	New line
<code>\t</code>	Tab
<code>\'</code>	Single quote
<code>\"</code>	Double quote
<code>\\</code>	Backslash

```
System.out.println("Hello\nWorld");
```

4. Boolean Literals

Only **two values**:

```
boolean isJavaFun = true;
boolean isRaining = false;
```

5. String Literals (Non-Primitive)

A sequence of characters enclosed in **double quotes**:

```
String name = "John";
String greeting = "Hello, World!";
```



Stored as an object of the `String` class in the **String pool**

6. Null Literal

`null` is a **special literal** used to represent the absence of a reference:

```
String str = null;
Student student = null;
```

Complete Example Code

```
public class LiteralExample {  
    public static void main(String[] args) {  
        int dec = 100;           // Decimal literal  
        int oct = 0123;         // Octal (83)  
        int hex = 0x7B;         // Hex (123)  
        int bin = 0b1010;       // Binary (10)  
        long big = 10000000000L; // Long literal  
        float pi = 3.14f;       // Float literal  
        double price = 9.99e2;   // 999.0 (scientific)  
        char c = 'A';  
        char unicodeChar = '\u0042'; // 'B'  
        boolean isJavaFun = true;  
        String message = "Hello\nJava!";  
        String str = null;  
        System.out.println(message);  
    }  
}
```



Summary Table

Type	Literal Examples
Integer	100, 0x1F, 075, 0b1010
Long	100000000000L
Float	3.14f, 2e2f
Double	9.99, 1.5e3
Char	'A', '\u0041', '9'
Boolean	true, false
String	"Java", "Hello\nWorld"
Null	null (for objects only)

OPERATORS

What is an Operator in Java?

An **operator** is a symbol that performs operations on variables and values.

Java provides a rich set of operators to manipulate variables, control flow, and perform computations.

Types of Operators in Java

Java supports **7 main types** of operators:



1. Arithmetic Operators

Used to perform **basic math operations**.

Operator	Meaning	Example	Result
+	Addition	5 + 3	8
-	Subtraction	5 - 2	3
*	Multiplication	4 * 2	8
/	Division	10 / 2	5
%	Modulus	10 % 3	1

 **Integer division** gives **quotient** without decimals.

2. Relational (Comparison) Operators

Used to **compare values** and return boolean results.

Operator	Meaning	Example	Result
<code>==</code>	Equal to	<code>5 == 5</code>	<code>true</code>
<code>!=</code>	Not equal to	<code>5 != 3</code>	<code>true</code>
<code>></code>	Greater than	<code>5 > 3</code>	<code>true</code>
<code><</code>	Less than	<code>2 < 5</code>	<code>true</code>
<code>>=</code>	Greater than or equal	<code>5 >= 5</code>	<code>true</code>
<code><=</code>	Less than or equal	<code>4 <= 6</code>	<code>true</code>

3. Logical Operators

Used to combine **boolean expressions**.

Operator	Meaning	Example	Result
<code>&&</code>	Logical AND	<code>true && false</code>	<code>false</code>
<code> </code>	Logical OR	<code>true false</code>	<code>true</code>
<code>!</code>	Logical NOT	<code>!true</code>	<code>false</code>

🧠 Used heavily in conditional statements like `if`, `while`.

4. Bitwise Operators

Operate on bits, **used for low-level operations**.

Operator	Meaning	Example	Result
&	Bitwise AND	5 & 3	1
	Bitwise OR	5 3	7 (111)
^	Bitwise XOR	5 ^ 3	6
~	Bitwise Complement	~5	-6
<<	Left Shift	5 << 1	10
>>	Right Shift	5 >> 1	2
>>>	Unsigned Right Shift	-1 >>> 1	large int

5. Assignment Operators

Used to **assign values** to variables.

Operator	Meaning	Example
=	Assign	a = 10
+=	Add and assign	a += 5 (a = a + 5)
-=	Subtract and assign	a -= 2
*=	Multiply and assign	a *= 3
/=	Divide and assign	a /= 2
%=	Modulus and assign	a %= 3

6. Unary Operators

Operate on **only one operand**.

Operator	Meaning	Example	Result
+	Unary plus (positive)	+5	5
-	Unary minus (negative)	-a	Negative of a
++	Increment	a++, ++a	a + 1
--	Decrement	a--, --a	a - 1
!	Logical NOT	!true	false

➡ ++a is **pre-increment**, a++ is **post-increment**

7. Ternary Operator (?:)

Java's only **conditional operator**.

Used as a shorthand for **if-else**.

`int max = (a > b) ? a : b;` Means: If `a > b`, then `max = a`; else `max = b`.



Summary Table

Type	Operators Used
Arithmetic	+, -, *, /, %
Relational	==, !=, <, >, <=, >=
Logical	&&, `
Bitwise	&, `
Assignment	=, +=, -=, *=, /=, %= %=
Unary	+, -, ++, --, !
Ternary	? :

Example Code Using All Types

```
public class OperatorExample {  
    public static void main(String[] args) {  
        int a = 10, b = 5;  
  
        // Arithmetic  
        System.out.println("Add: " + (a + b));  
  
        // Relational  
        System.out.println("Equal: " + (a == b));  
  
        // Logical  
        boolean x = true, y = false;  
        System.out.println("AND: " + (x && y));  
  
        // Bitwise  
        System.out.println("Bitwise OR: " + (a | b));  
  
        // Assignment  
        a += 5;  
        System.out.println("After += : " + a);  
  
        // Unary  
        int c = 3;  
        System.out.println("++c: " + (++c));  
  
        // Ternary  
        int max = (a > b) ? a : b;  
        System.out.println("Max: " + max);  
    }  
}
```

! Common Mistakes

- `=` is **assignment**, not **comparison**. Use `==` to compare.
- `a++` vs. `++a`: post-increment vs. pre-increment.
- `&&` and `||` are **short-circuit operators**.
- Bitwise operators are **not** the same as logical operators.

Type Conversion / Type Casting

Type Conversion:

Type Conversion (aka **Type Casting**) is the process of converting a variable from **one data type to another**.

For example:

```
int x = 10;
```

```
double y = x; // int is converted to double
```



Types of Type Conversion

1. Implicit Conversion (Automatic)

- ➡ Smaller type → Larger type
- ➡ Done **automatically** by Java
- ➡ No data loss

```
byte → short → int → long → float → double
```

```
char → int
```

◆ Example:

```
int a = 100;
```

```
double d = a; // Implicit casting: int → double
```

```
System.out.println(d); // Output: 100.0
```

2. Narrowing Conversion (Explicit / Manual)

- ➡ Larger type → Smaller type
- ➡ Must be **manually specified**
- ➡ May lose data or precision

```
double → float → long → int → short → byte
```


◆ Syntax:

type **variableName** = (**type**) **value**;

◆ Example:

```
double d = 45.89;

int i = (int) d; // Narrowing conversion

System.out.println(i); // Output: 45 (decimal part lost)
```



Type Conversion Chart

byte → short → int → long → float → double

↑ ↑ ↑ ↑

char (narrowing for reverse direction)

Type Conversion with Characters

```
char ch = 'A';

int ascii = ch; // Implicit

System.out.println(ascii); // Output: 65
```

```
int code = 97;

char c = (char) code; // Explicit

System.out.println(c); // Output: a
```

Type Conversion with Expressions

When mixing data types in expressions:

- Java **automatically promotes** smaller types
- Final result type = **largest type in the expression**

◆ Example:

```
int a = 5;

double b = 6.2;

double result = a + b; // int is converted to double
```



Type Conversion Between Non-Compatible Types

You **cannot** convert:

- `String` ↔ `int` directly (needs parsing)
- `boolean` ↔ any other type (not allowed)

◆ Converting String to Primitive Types

```
String s = "123";

int num = Integer.parseInt(s); // String → int
```

Other methods:

- `Double.parseDouble("3.14")`
- `Boolean.parseBoolean("true")`

◆ Converting Primitive Types to String

```
int x = 10;

String s = String.valueOf(x);
```

! Common Mistakes:

Mistake	Why It's Wrong
<code>int x = 10.5;</code>	Error – narrowing without casting
<code>String s = 10;</code>	Error – must convert explicitly
<code>boolean b = (boolean) 1;</code>	✗ Not allowed in Java
<code>int a = 130; byte b = (byte) a;</code>	! Overflow (b = -126)

Example Code: All Type Casting

```

public class TypeCastingExample {
    public static void main(String[] args) {
        // Widening
        int a = 20;
        double d = a;
        System.out.println("Widening int to double: " + d);

        // Narrowing
        double x = 10.75;
        int y = (int) x;
        System.out.println("Narrowing double to int: " + y);

        // Character to int
        char ch = 'B';
        int ascii = ch;
    }
}

```

```
System.out.println("Char to int: " + ascii);

// String to int

String s = "123";

int num = Integer.parseInt(s);

System.out.println("String to int: " + num);

// int to String

String str = String.valueOf(num);

System.out.println("int to String: " + str);

}

}
```

Module 03: Control Flow Statements

1. if Statement

Syntax:

```
if (condition) {  
    // code to execute if condition is true  
}
```

Explanation:

- The **if** statement checks a condition.
- If the condition is **true**, it executes the block inside.
- If **false**, it skips the block.

Example:

```
int age = 20;  
if (age >= 18) {  
    System.out.println("You are eligible to vote.");  
}
```

2. if-else Statement

Syntax:

```
if (condition) {  
    // if condition is true  
} else {  
    // if condition is false  
}
```

Explanation:

- If condition is true → first block runs.
- Otherwise → **else** block runs.

Example:

```
int marks = 40;
if (marks >= 50) {
    System.out.println("You passed.");
} else {
    System.out.println("You failed.");
}
```

3. if-else-if Ladder

Syntax:

```
if (condition1) {
    // block1
} else if (condition2) {
    // block2
} else if (condition3) {
    // block3
} else {
    // default block
}
```

Explanation:

- Used to check multiple conditions one by one.
- Only the first true condition's block executes.

Example:

```
int marks = 85;

if (marks >= 90) {
    System.out.println("Grade A");
} else if (marks >= 80) {
    System.out.println("Grade B");
} else if (marks >= 70) {
    System.out.println("Grade C");
} else {
    System.out.println("Grade D");
}
```

4. switch Statement

Syntax:

```
switch (expression) {  
    case value1:  
        // code block  
        break;  
    case value2:  
        // code block  
        break;  
    ...  
    default:  
        // default code  
}
```

Explanation:

- Compares **expression** against multiple **case** values.
- **break** exits the switch after one match.
- **default** executes if no case matches.

Example:

```
int day = 3;  
switch (day) {  
    case 1:  
        System.out.println("Monday");  
        break;  
    case 2:  
        System.out.println("Tuesday");  
        break;  
    case 3:  
        System.out.println("Wednesday");  
        break;  
    default:  
        System.out.println("Invalid day");  
}
```

✓ Without **break** (Fall-through behavior):

```
int num = 2;
switch (num) {
    case 1:
        System.out.println("One");
    case 2:
        System.out.println("Two");
    case 3:
        System.out.println("Three");
}
```

Output:

Two
Three

This happens because no **break** is used.

5. Ternary Operator ? :

Syntax:

`variable = (condition) ? value_if_true : value_if_false;`

Explanation:

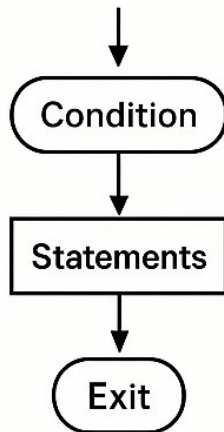
- Short-hand **if-else** in one line.
- Used to assign a value based on condition.

Example:

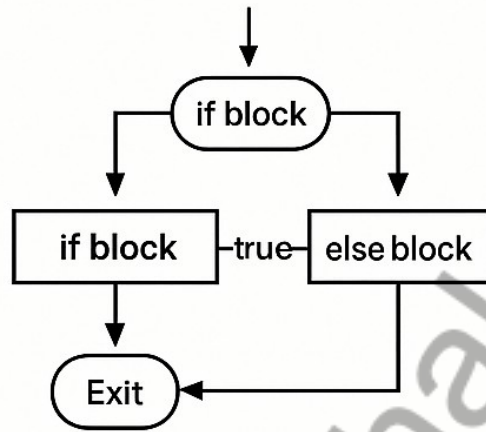
```
int age = 17;
String result = (age >= 18) ? "Adult" : "Minor";
System.out.println(result);
```

Output:
Minor

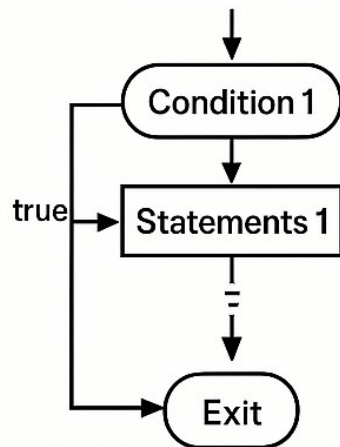
if



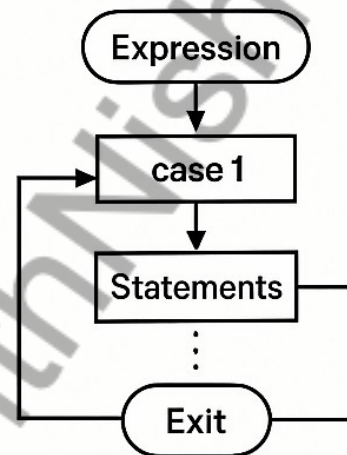
if-else



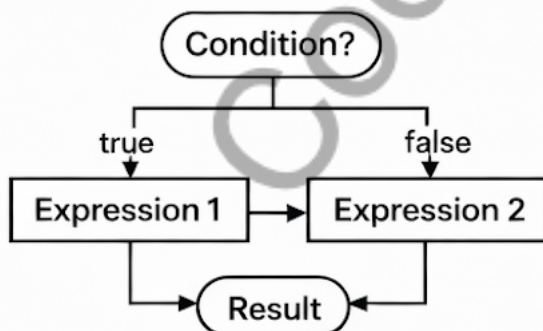
if-else-if Ladder



switch-case



Ternary Operator



Loops in JAVA

Problem Without Loops:

Imagine you want to print "Hello World" 5 times.

```
System.out.println("Hello World");
System.out.println("Hello World");
System.out.println("Hello World");
System.out.println("Hello World");
System.out.println("Hello World");
```

This approach is:

- **Repetitive**
- **Hard to modify** (what if we want 1000 prints?)
- **Not scalable or efficient**

What Are Loops?

- A **loop** is a programming structure that repeats a block of code **as long as a condition is true**.
- Instead of writing the same code multiple times, you use a loop to **execute once, many times**.

There are multiple loops in JAVA:

1. While loop
2. Do while loop
3. For loop
4. For each loop.(we will learn it once we will learn array)

1. while Loop in Java

What is a while loop?

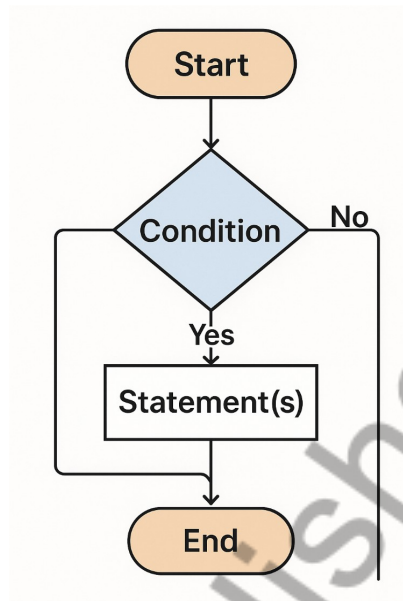
A while loop **repeats a block of code as long as the given condition is true**.

Use it **when you don't know how many times** you want to run the loop — but you want to keep running **until** something happens.

Syntax:

```
while (condition) {
    // Code to be repeated
}
```

- **Condition** is checked **before** the loop body runs.
- If the condition is **false at the start**, the loop runs **0 times**.
- As long as the condition remains **true**, the loop will keep running.



Example 1: Print numbers 1 to 5

```
public class WhileExample1 {  
    public static void main(String[] args) {  
        int i = 1;  
        while (i <= 5) {  
            System.out.println(i);  
            i++;  
        }  
    }  
}
```

Explanation:

- Starts at $i = 1$
- Prints 1 2 3 4 5
- Stops when $i > 5$

Example 2: Keep asking user until correct password is entered

```
import java.util.Scanner;

public class WhileExample2 {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        String password = "";

        while (!password.equals("admin123")) {
            System.out.print("Enter password: ");
            password = sc.nextLine();
        }

        System.out.println("Access granted!");
    }
}
```

Explanation:

- Keeps asking for input **until** user types "admin123"
- Useful in login systems, retry prompts, etc.

2. do-while Loop in Java:

What is a do-while loop?

A do-while loop is similar to a while loop, but with one major difference:

The loop body runs at least once, before checking the condition.

Syntax:

```
do {
    // Code to be executed
} while (condition);
```

- The code inside do block executes first.
- Then the condition is checked.
- If the condition is true, loop continues.
- If condition is false, loop stops.

Example 1: Print numbers 1 to 5 using do-while

```
public class DoWhileExample1 {  
    public static void main(String[] args) {  
        int i = 1;  
        do {  
            System.out.println(i);  
            i++;  
        } while (i <= 5);  
    }  
}
```

Output:

```
1  
2  
3  
4  
5
```

Example 2: Menu-Driven Program (Run at Least Once):

```
import java.util.Scanner;  
  
public class DoWhileExample2 {  
    public static void main(String[] args) {  
        Scanner sc = new Scanner(System.in);  
        int choice;  
  
        do {  
            System.out.println("Menu:");  
            System.out.println("1. Say Hello");  
            System.out.println("2. Say Bye");  
            System.out.println("3. Exit");  
            System.out.print("Enter choice: ");  
            choice = sc.nextInt();  
  
            if (choice == 1)  
                System.out.println("Hello!");  
            else if (choice == 2)  
                System.out.println("Bye!");  
        } while (choice != 3);  
  
        System.out.println("Exited!");  
    }  
}
```

- The menu is shown **at least once**, even if the user enters 3 right away.
- Perfect for menu-based logic or retry-systems.

3. for Loop in Java

What is a for loop?

A for loop is used when you **know in advance how many times** you want to run the loop.

It is best for **count-controlled** repetition.

Syntax:

```
for (initialization; condition; inc/dec) {  
    // Code to repeat  
}
```

Components of for loop:

Part	Purpose
Initialization	Sets the starting point (runs once before the loop starts)
Condition	Checked before each iteration ; loop continues if true
Update	Changes the loop variable (runs after each iteration)

Example 1: Print numbers from 1 to 5

```
public class ForLoopExample1 {  
    public static void main(String[] args) {  
        for (int i = 1; i <= 5; i++) {  
            System.out.println(i);  
        }  
    }  
}
```

Output:

```
1  
2  
3  
4  
5
```

Explanation:

- `i = 1` (initialization)
- Checks `i <= 5` (condition)
- Prints and then `i++` (update)
- Stops when `i = 6`

Example 2: Print even numbers from 2 to 10

```
public class ForLoopExample2 {  
    public static void main(String[] args) {  
        for (int i = 2; i <= 10; i += 2) {  
            System.out.print(i + " ");  
        }  
    }  
}
```

Output:

2 4 6 8 10

Explanation:

- Starts from 2
- Increments by 2
- Checks condition `i <= 10` each time

Summary

- for loop is great for **counted iterations**
- Use it when you know the **start, end, and step**
- Very common in loops involving **arrays, strings, patterns, mathematical calculations**, etc.

OOPS BASICS

1. What is Object-Oriented Programming (OOP)?

OOP is a programming paradigm where the code is organized into **objects** and **classes**. It helps build modular, reusable, and scalable software.

2. Classes in Java

What is a Class?

A class is a **blueprint/template** for creating objects. It defines **variables** (data/state) and **methods** (behavior) inside it.

Syntax:

```
class ClassName {  
    // variables (fields)  
    // methods  
}
```

Example 1: A Car class

```
class Car {  
    String brand = "Toyota";  
    int speed = 120;  
  
    void displayInfo() {  
        System.out.println(brand + " runs at " + speed + " km/h");  
    }  
}
```

This class defines a car with brand and speed and a method to display info.

Example 2: A Student class

```
class Student {  
    String name;  
    int age;  
}
```

This is a basic class with two fields. No methods yet.

3. Objects in Java

What is an Object?

An object is an **instance** of a class.

It has real existence in memory, unlike a class (which is abstract).

Syntax:

```
ClassName obj = new ClassName();
```

Example 1: Create a Car object

```
public class Main {  
    public static void main(String[] args) {  
        Car myCar = new Car();  
        myCar.displayInfo(); // Output: Toyota runs at 120 km/h  
    }  
}
```

Example 2: Set values in Student object:

```
public class Main {  
    public static void main(String[] args) {  
        Student s1 = new Student();  
        s1.name = "Aman";  
        s1.age = 20;  
  
        System.out.println(s1.name + " is " + s1.age + " years old.");  
    }  
}
```

4. Methods in Java

What is a Method?

A method is a block of code that performs a task. It defines **behavior** of an object.

Syntax:

```
<access modifiers> <returnType> <methodName>(parameters) {  
    // code  
}
```

Method Types:

- Void Method: Doesn't return anything
- Return Method: Returns a value
- Parameterised Method: Accepts arguments

Example 1: Method with no return

```
class Calculator {  
    void greet() {  
        System.out.println("Welcome to Calculator!");  
    }  
}
```

Example 2: Method with return value

```
class Calculator {  
    int add(int a, int b) {  
        return a + b;  
    }  
}
```

5. Method Overloading

What is Method Overloading?

Method overloading is when multiple methods have the same name but with:

- Different number of parameters
- OR different types of parameters

It's a part of compile-time polymorphism in OOP.

Example 1: Method with different number of parameters

```
class Printer {  
    void print(String msg) {  
        System.out.println(msg);  
    }  
  
    void print(String msg, int times) {  
        for (int i = 0; i < times; i++) {  
            System.out.println(msg);  
        }  
    }  
}
```

Example 2: Method with different types of parameters

```
class MathUtil {  
    int multiply(int a, int b) {  
        return a * b;  
    }  
  
    double multiply(double a, double b) {  
        return a * b;  
    }  
}
```

Summary Table

Concept	Description	Key Feature
Class	Blueprint of object	Contains fields + methods
Object	Instance of a class	Real entity with memory
Method	Action/behavior inside class	Performs logic or operation
Method Overloading	Same method name, different signature	Compile-time polymorphism

Memory Management and Arrays

What is Stack Memory?

- It is a temporary memory space used for:
 - Method/function calls
 - Local variables
 - Method parameters

Key Features:

Feature	Description
 Stores	Local variables and method call frames
 LIFO	Last-In-First-Out structure
 Fast	Very fast access and automatically managed
 No Object Storage	Does not store objects (only references)

How It Works:

Every time a **method is called**, a **new stack frame** is created:

- Stores **parameters, local variables**
- Gets **deleted automatically** after method execution ends

2. Heap Memory

- It is the memory area where **objects** and **instance variables** are stored.

Key Features:

Feature	Description
 Stores	Objects (instances of classes)
 Shared	Accessible by all threads
 Garbage Collected	Java automatically removes unused objects
 Slower	Slower than stack but used for long-term memory

3. Instance Variables & Heap

What are Instance Variables?

- Variables declared **inside a class but outside any method**
- Every **object gets its own copy**
- Stored in **Heap** as part of the object

Visualization of Stack vs Heap

Let's understand through an **example**:

Example 1: Instance variables + method

```
class Person {
    String name; // instance variable (in heap)

    void setName(String newName) {
        name = newName; // local variable newName is in stack
    }
}

public class Main {
    public static void main(String[] args) {
        Person p = new Person(); // 'p' ref in stack, object in heap
        p.setName("Nishchal");
    }
}
```

Stack:

- main() stack frame
 - reference p
- setName() stack frame
 - parameter newName = "Nishchal"

Heap:

- Person object
 - instance variable name = "Nishchal"

After setName() ends, newName is removed from stack, but name remains in heap.

Example 2: Multiple objects, multiple methods

```
class Account {  
    int balance; // instance variable  
  
    void deposit(int amount) {  
        balance += amount;  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {  
        Account a1 = new Account(); // Object1  
        Account a2 = new Account(); // Object2  
  
        a1.deposit(100); // call stack for deposit()  
        a2.deposit(200); // separate object, separate memory in heap  
    }  
}
```

Memory Layout:

Stack:

- main() → a1, a2 (references)
- deposit(100) → local amount (stack frame)
- deposit(200) → another stack frame

Heap:

- a1 object → balance = 100
- a2 object → balance = 200

Both objects live **independently** in heap. Stack is used temporarily during function calls.

Summary Table

Aspect	Stack	Heap
Used for	Method calls, local vars	Objects & instance variables
Scope	Temporary (per method call)	Until object is unreachable
Managed by	JVM automatically	JVM via Garbage Collector
Access Speed	Fast	Slower
Stores	Primitives, references, call stacks	Actual object data

ARRAY

An **array** is a **container object** that holds **multiple elements of the same type** in a **contiguous block of memory**.

1. One-Dimensional Array (1D Array):

Syntax:

```
dataType[] arrayName = new dataType[size];
```

Declaration & Initialization:

```
int[] arr = new int[5]; // Declaration with size  
int[] arr = {1, 2, 3, 4, 5}; // Declaration + Initialization
```

Traversing a 1D Array:

```
for (int i = 0; i < arr.length; i++) {  
    System.out.println(arr[i]);  
}
```

It will print all the elements of array.

Example:

```
public class OneDArray {  
    public static void main(String[] args) {  
        int[] marks = {90, 85, 78, 92};  
        for (int mark : marks) {  
            System.out.println(mark);  
        }  
    }  
}
```

Array Operations:

A. Insertion in 1D Array

```
int[] arr = new int[5];  
arr[0] = 10; // Insert at index 0
```

B. Deletion (Manually shift elements)

```
int[] arr = {1, 2, 3, 4, 5};  
int indexToDelete = 2; // Delete 3rd element  
  
for (int i = indexToDelete; i < arr.length - 1; i++) {  
    arr[i] = arr[i + 1];  
}  
// Last element becomes duplicate or unused  
arr[arr.length - 1] = 0; // Optional cleanup
```

C. Traversal

```
for (int val : arr) {  
    System.out.print(val + " ");  
}
```

D. Searching an element

```
int search = 30;  
for (int i = 0; i < arr.length; i++) {  
    if (arr[i] == search) {  
        System.out.println("Found at index " + i);  
        break;  
    }  
}
```

Java provides some helper methods from `java.util.Arrays`:

```
import java.util.Arrays;
```

Built-in Methods (1D Arrays)

Method	Description
<code>Arrays.toString(arr)</code>	Print 1D array
<code>Arrays.sort(arr)</code>	Sort ascending
<code>Arrays.copyOf(arr, newLength)</code>	Resize array
<code>Arrays.equals(arr1, arr2)</code>	Compare arrays
<code>Arrays.binarySearch(arr, key)</code>	Returns index (sorted array)
<code>Arrays.fill(arr, val)</code>	Fill with value

Examples:

```
int[] arr = {5, 3, 1, 4};  
Arrays.sort(arr);           // [1, 3, 4, 5]  
System.out.println(Arrays.toString(arr));
```

5. Two-Dimensional Array (2D Arrays)

Declaration:

```
int[][] matrix = new int[3][3]; // 3x3 matrix
```

Initialization:

```
int[][] mat = {  
    {1, 2, 3},  
    {4, 5, 6},  
    {7, 8, 9}  
};
```

Traversing a 2D Array:

```
for (int i = 0; i < mat.length; i++) {  
    for (int j = 0; j < mat[i].length; j++) {  
        System.out.print(mat[i][j] + " ");  
    }  
    System.out.println();  
}
```

Example: Sum of all elements in 2D array

```
int sum = 0;  
for (int[] row : mat) {  
    for (int num : row) {  
        sum += num;  
    }  
}  
System.out.println("Sum = " + sum);
```

Strings and Wrapper Classes

1. What is a String in Java?

- A **String** is a **sequence of characters**.
- Strings in Java are **immutable** (once created, cannot be changed).
- Stored in a **String pool** (if created with `""`) or heap (if created with `new`).

Declaration:

```
String s1 = "Java";           // String pool
String s2 = new String("Java"); // Heap memory
```

2. Common String Methods

Importing is not needed for `String`, it's part of `java.lang`.

=> Character & Length

Method	Description	Example
<code>length()</code>	Number of characters	<code>"Java".length()</code> → 4
<code>charAt(i)</code>	Char at index i	<code>"Java".charAt(1)</code> → 'a'
<code>toCharArray()</code>	Converts to char array	<code>"Hi".toCharArray()</code>

=> Case Conversion

Method	Description	Example
<code>toLowerCase()</code>	Converts to lowercase	<code>"Java".toLowerCase()</code> → java
<code>toUpperCase()</code>	Converts to uppercase	<code>"java".toUpperCase()</code> → JAVA

=> Comparison

Method	Description	Example
<code>equals(str)</code>	Content equals (case-sensitive)	<code>"a".equals("a") → true</code>
<code>equalsIgnoreCase(str)</code>	Ignores case	<code>"A".equalsIgnoreCase("a")</code>
<code>compareTo(str)</code>	Lexicographic compare	<code>"a".compareTo("b") → -1</code>

=> Modification

Method	Description	Example
<code>replace(a, b)</code>	Replaces a with b	<code>"Java".replace('a','o') → Jovo</code>
<code>substring(i)</code>	Substring from index	<code>"Hello".substring(2) → llo</code>
<code>substring(i, j)</code>	From i to j-1	<code>"Hello".substring(1,4) → ell</code>
<code>trim()</code>	Removes leading/trailing spaces	<code>" Hi ".trim() → Hi</code>

=> Searching

Method	Description	Example
<code>indexOf(ch)</code>	Index of first match	<code>"banana".indexOf('a') → 1</code>
<code>lastIndexOf(ch)</code>	Index of last match	<code>"banana".lastIndexOf('a') → 5</code>
<code>contains(str)</code>	Checks presence	<code>"Java".contains("va") → true</code>
<code>startsWith(str)</code>	Checks prefix	<code>"Java".startsWith("Ja")</code>
<code>endsWith(str)</code>	Checks suffix	<code>"Java".endsWith("va")</code>

=> Joining & Splitting

Method	Description	Example
<code>concat(str)</code>	Joins strings	<code>"Hi".concat("Bye")</code>
<code>split(regex)</code>	Splits into array	<code>"a,b,c".split(",")</code>

Example: Using String Methods

```
public class Test {  
    public static void main(String[] args) {  
        String s = " Java is awesome ";  
        System.out.println(s.trim().toUpperCase().replace("AWESOME", "powerful"));  
    }  
}  
// Output: JAVA IS POWERFUL
```

Part 1: Why Strings are Immutable in Java What is Immutability?

An object is **immutable** if its state (i.e., the data it holds) cannot be changed after it is created.

Why are Strings Immutable?

1. **Security:** Strings are often used for sensitive data like usernames, passwords, and network URLs. Immutable strings ensure this data can't be altered.
2. **String Pooling:** Java uses the **String pool** to store string literals. Immutability allows reuse without fear of modification.
3. **Thread-Safety:** Immutable objects are inherently thread-safe. Multiple threads can use the same string without synchronization.
4. **HashCode Caching:** Strings are often used as keys in hash-based collections (e.g., HashMap). Their hashcode is cached on first call for performance.
5. **Class Loading:** Strings are used in class loading. Immutable strings help in maintaining the integrity of class paths.

Example:

```
String s1 = "Hello";  
String s2 = s1;  
s1 = s1 + " World";  
  
System.out.println(s1); // Hello World  
System.out.println(s2); // Hello
```

`s1` was not modified; a **new object** was created.

Part 2: StringBuffer and StringBuilder

Java provides two mutable alternatives to `String`:

- **StringBuffer** (Thread-safe)
- **StringBuilder** (Not Thread-safe but faster)

1. StringBuffer

- Mutable (can change content)
- Thread-safe (synchronized methods)
- Slightly slower than StringBuilder

```
StringBuffer sb = new StringBuffer("Hello");
```

Common Methods of StringBuffer

Method	Description
<code>append(String s)</code>	Adds string to the end
<code>insert(int offset, String s)</code>	Inserts string at specified index
<code>replace(int start, int end, String str)</code>	Replaces characters between start and end
<code>delete(int start, int end)</code>	Deletes characters from start to end
<code>reverse()</code>	Reverses the characters
<code>capacity()</code>	Returns current capacity
<code>ensureCapacity(int min)</code>	Increases capacity
<code>charAt(int index)</code>	Returns character at index
<code>setCharAt(int index, char ch)</code>	Sets character at index
<code>toString()</code>	Converts to String

Example

```
StringBuffer sb = new StringBuffer("Java");  
sb.append(" Rocks"); // Java Rocks  
sb.insert(4, " is"); // Java is Rocks  
sb.replace(5, 7, "are"); // Java are Rocks  
sb.delete(5, 9);      // Java Rocks  
sb.reverse();         // skcoR avaJ
```


2. StringBuilder

Features:

- Mutable
- Not thread-safe (no synchronization)
- Faster performance than StringBuffer

Creating StringBuilder

```
StringBuilder sb = new StringBuilder("Hello");
```

Methods of StringBuilder

Same as `StringBuffer`. Only difference is performance and synchronization.

Method	Description
<code>append(String s)</code>	Adds string to the end
<code>insert(int offset, String s)</code>	Inserts string at index
<code>replace(int start, int end, String str)</code>	Replaces characters
<code>delete(int start, int end)</code>	Deletes part of string
<code>reverse()</code>	Reverses the string
<code>capacity()</code>	Returns capacity
<code>ensureCapacity(int min)</code>	Ensures capacity
<code>charAt(int index)</code>	Gets character at index
<code>setCharAt(int index, char ch)</code>	Sets character
<code>toString()</code>	Converts to String



Example

```
StringBuilder sb = new StringBuilder("Flip");  
sb.append("kart");    // Flipkart  
sb.insert(4, "Zone"); // FlipZonekart  
sb.replace(4, 8, "Deal"); // FlipDealkart  
sb.delete(4, 8);      // Flipkart  
sb.reverse();         // trakpilF
```

Difference Table

Feature	String	StringBuffer	StringBuilder
Mutability	Immutable	Mutable	Mutable
Thread-safe	Yes	Yes	✗ No
Performance	Medium	Slower	Fast
Use Case	Fixed text	Multithreaded apps	Single-threaded apps

When to Use What?

- Use **String** when:
 - You don't need to modify the text.
 - Memory-efficient string reuse is important.
- Use **StringBuffer** when:
 - You need to modify strings in **multi-threaded** environment.
- Use **StringBuilder** when:
 - You need to modify strings in a **single-threaded** context for **better performance**.

WRAPPER CLASSES

What are Wrapper Classes?

Wrapper classes are used to **wrap primitive data types** into objects. Java provides a wrapper class for each primitive type.

Primitive	Wrapper Class
byte	Byte
short	Short
int	Integer
long	Long
float	Float
double	Double
char	Character
boolean	Boolean

Why Use Wrapper Classes?

1. **Collections** (like `ArrayList`) can't store primitives—only objects.
2. **Utility methods**: Wrapper classes have utility methods like `parseInt()`, `toString()`, etc.
3. **Object-based APIs**: Some APIs require objects instead of primitives.
4. **Null values**: Objects can be null, primitives cannot.
5. **Type Conversion**: Help in converting data between different formats.

Commonly Used Methods

Here are some popular methods available in wrapper classes:

Example: `Integer` class methods

```
Integer x = Integer.valueOf("10"); // Converts String to Integer
int y = Integer.parseInt("123"); // Converts String to int
```

Method	Description
<code>valueOf(String s)</code>	Returns wrapper object from string
<code>parseInt(String s)</code>	Returns primitive <code>int</code> from string
<code>toString()</code>	Converts value to string
<code>compareTo()</code>	Compares two wrapper objects
<code>equals(Object obj)</code>	Compares wrapper object values
<code>intValue(), doubleValue()</code>	Extracts primitive value



Boxing and Unboxing

1. Boxing

Manual conversion of a primitive into its wrapper object.

```
int a = 10;
Integer boxedA = Integer.valueOf(a); // Boxing
```

2. Unboxing

Manual conversion of wrapper object back to primitive.

```
Integer boxedA = Integer.valueOf(10);
int a = boxedA.intValue(); // Unboxing
```



Autoboxing and Auto-unboxing

1. Autoboxing

Automatic conversion by the compiler from a primitive to its wrapper class object.

```
int x = 5;
Integer obj = x; // Autoboxing
```

2. Auto-unboxing

Automatic conversion from wrapper class to primitive.

```
Integer obj = 100;
int x = obj; // Auto-unboxing
```

Full Example:

```
import java.util.ArrayList;

public class WrapperDemo {
    public static void main(String[] args) {
        // Autoboxing
        Integer a = 10;
        Double b = 12.5;

        // Auto-unboxing
        int c = a;
        double d = b;

        // Using in collections
        ArrayList<Integer> list = new ArrayList<>();
        list.add(5); // Autoboxing
        list.add(10);

        int sum = list.get(0) + list.get(1); // Auto-unboxing
        System.out.println("Sum: " + sum);
    }
}
```

Differences Table

Concept	Definition	Example
Boxing	Manual primitive → object	<code>Integer i = Integer.valueOf(5);</code>
Autoboxing	Automatic primitive → object	<code>Integer i = 5;</code>
Unboxing	Manual object → primitive	<code>int i = iObj.intValue();</code>
Auto-unboxing	Automatic object → primitive	<code>int i = iObj;</code>

Summary:

- Wrapper classes wrap primitives into objects.
- **Boxing/Unboxing** are manual conversions.
- **Autoboxing/Auto-unboxing** are compiler features.
- Essential for **Collections** and **Object-oriented** operations.

CodeWithNishchal

Static Members and Encapsulation

1. Static Variable

Definition:

A **static variable** is a class-level variable shared by **all instances** of the class. It is initialized **only once** at the time of class loading. **Static variables** in Java are stored in the **Method Area** of the **JVM memory**. **This is a special separate block of memory.**

Characteristics:

- Belongs to the **class**, not to any specific object.
- All objects **share the same copy** of the static variable.
- Can be accessed using:
 - `ClassName.variableName` (recommended)
 - `object.variableName` (valid but not preferred)

Syntax:

```
class MyClass {  
    static int count = 0; // static variable  
  
    MyClass() {  
        count++; // increment shared variable  
    }  
}
```

Example:

```
class Counter {  
    static int count = 0;  
  
    Counter() {  
        count++;  
        System.out.println("Object created. Count = " + count);  
    }  
  
    public static void main(String[] args) {  
        new Counter();  
        new Counter();  
        new Counter();  
    }  
}
```

Output:

Object created. Count = 1
Object created. Count = 2
Object created. Count = 3

2. Static Block

Definition:

A **static block** is a block of code that runs **once** when the class is loaded into memory, **before** any object is created or static methods are called.

Characteristics:

- Used for **static initialization** (e.g., setting up static variables, loading files/resources).
- Executes **only once**, when the class is first loaded.
- Useful for **complex initialization logic**.

Syntax:

```
class MyClass {  
    static int data;  
  
    static {  
        data = 100;  
        System.out.println("Static block executed. Data = " + data);  
    }  
}
```

Example:

```
class StaticBlockExample {  
    static int a;  
    static {  
        a = 10;  
        System.out.println("Static block executed. Value of a: " + a);  
    }  
  
    public static void main(String[] args) {  
        System.out.println("Main method started.");  
        System.out.println("Value of a = " + a);  
    }  
}
```

Output:

Static block executed. Value of a: 10
Main method started.
Value of a = 10

3. Static Method

Definition:

A **static method** belongs to the class rather than any object of the class.

Characteristics:

- Can be called **without creating an object**.
- Can **only access static data** and call **other static methods**.
- Cannot use **this** or **super** keywords.

Syntax:

```
class MyClass {  
    static void display() {  
        System.out.println("This is a static method.");  
    }  
}
```

Example:

```
class StaticMethodExample {  
    static int a = 5;  
  
    static void show() {  
        System.out.println("Static method called. a = " + a);  
    }  
  
    public static void main(String[] args) {  
        StaticMethodExample.show(); // recommended  
    }  
}
```

Output:

Static method called. a = 5

✓ Summary Table:

Feature	Static Variable	Static Method	Static Block
Belongs to	Class	Class	Class
Runs	When class loaded	When explicitly called	When class loaded
Access via	<code>ClassName.var</code>	<code>ClassName.method()</code>	Auto-run
Object required?	✗ No	✗ No	✗ No
Access instance members?	✗ No	✗ No	✗ No

Encapsulation

Definition:

Encapsulation is the process of **wrapping data (variables)** and **methods (code)** together into a single unit — typically a **class** — and restricting direct access to some of the object's components.

It is a **fundamental OOP principle**.

✓ Key Goals of Encapsulation:

- **Protect data** from unauthorized access
- **Hide internal details** (data hiding)
- **Control how data is accessed or modified**
- Improves **code maintainability** and **security**

✓ How to Achieve Encapsulation in Java:

1. **Declare variables as `private`** (to hide them from outside access).
2. **Provide `public` getter and setter methods** to read or update the value.

Example:

```
class Student {  
    // Step 1: Private data members  
    private String name;  
    private int age;  
  
    // Step 2: Public getter method  
    public String getName() {  
        return name;  
    }  
  
    // Step 3: Public setter method  
    public void setName(String name) {  
        this.name = name;  
    }  
  
    public int getAge() {  
        return age;  
    }  
  
    public void setAge(int age) {  
        if (age > 0) { // basic validation  
            this.age = age;  
        }  
    }  
}
```

✓ Usage:

```
public class Main {  
    public static void main(String[] args) {  
        Student s = new Student();  
  
        s.setName("Ravi");  
        s.setAge(20);  
  
        System.out.println("Name: " + s.getName());  
        System.out.println("Age: " + s.getAge());  
    }  
}
```

=>Why Make Variables Private?

Making variables **private**:

- Prevents direct access or tampering from outside
- Enforces controlled access via methods
- Allows for **validation** before setting values (e.g., `age > 0`)

Benefits of Encapsulation:

Benefit	Explanation
Data Hiding	Internal data is hidden from outside classes
Improved Security	Only allowed methods can modify the data
Flexibility	You can change the internal code without affecting other classes
Easy to Maintain	Well-structured and clean code
Increased Control	You can add validation logic in setters

✗ Without Encapsulation (Bad Practice):

```
class Student {  
    public String name; // No protection  
    public int age;  
}
```

Anyone can access and modify data like:

```
Student s = new Student();  
s.name = "Ravi";  
s.age = -5; // ❌ no validation
```

Summary Table:

Feature	Encapsulation Practice
Variable Access	private
Method Access	public getters/setters
Direct field access	❌ Not allowed outside class
Control/Validation	✅ Possible in setter

Real-world Example:

Think of **ATM Machine**:

- You **press buttons** (public methods)
- You **cannot directly access** the internal logic (private data)
- ATM decides what you can or cannot do — this is **encapsulation**

this Keyword in Java

Definition:

In Java, **this** is a **reference variable** that refers to the **current object** of the class — i.e., the object whose method or constructor is being executed.

Why Use **this**?

It helps distinguish between:

- **Instance variables** and
- **Local variables / parameters** with the **same name**.

Common Uses of **this** Keyword

1. To Refer to Current Class Instance Variable

When local variables (e.g., constructor or method parameters) have the **same name** as instance variables.

```
class Student {
    String name;

    Student(String name) {
        this.name = name; // this.name = instance variable, name = parameter
    }

    void print() {
        System.out.println("Name: " + name);
    }
}
```

2. To Invoke Current Class Method:

```
class Demo {  
    void show() {  
        System.out.println("Show method called");  
    }  
  
    void display() {  
        this.show(); // calling current class method  
    }  
}
```

3. To Invoke Current Class Constructor

We can call one constructor from another constructor in the same class using `this()`.

```
class Person {  
    String name;  
    int age;  
  
    Person() {  
        this("Unknown", 0); // calling parameterized constructor  
    }  
  
    Person(String name, int age) {  
        this.name = name;  
        this.age = age;  
    }  
  
    void print() {  
        System.out.println(name + " - " + age);  
    }  
}
```

CONSTRUCTOR

Constructor in Java

What is a Constructor?

A **constructor** in Java is a **special method** that is used to **initialize objects** of a class. It is **automatically called** when an object is created.

Characteristics of a Constructor:

Feature	Description
Name	Must be same as the class name
Return type	No return type , not even <code>void</code>
Called automatically	Yes, when an object is created
Overloading	✅ Yes, constructors can be overloaded
Inheritance	❌ Constructors are not inherited

✅ Syntax:

```
class ClassName {  
    ClassName() {  
        // constructor body  
    }  
}
```

1. Every time you create an object, the constructor is called.
2. It has no return type.
3. If we create 2 objects, the constructor will be called 2 times.

Java provides 2 Types of constructor:

1. Default Constructor

- A **constructor with no parameters**.
- Provided by the **compiler automatically** if no constructor is defined.
- Initializes instance variables with **default values**.

Example:

```
class Car {
    Car() {
        System.out.println("Default constructor called");
    }

    public static void main(String[] args) {
        Car c = new Car(); // default constructor is called
    }
}
```

2. Parameterized Constructor

- A constructor that **accepts parameters** to initialize objects with specific values.
- Allows you to create multiple objects with **different initial states**.

```
class Car {
    String brand;
    int year;

    // parameterized constructor
    Car(String b, int y) {
        brand = b;
        year = y;
    }

    void show() {
        System.out.println(brand + " - " + year);
    }

    public static void main(String[] args) {
        Car c1 = new Car("Honda", 2020);
        Car c2 = new Car("Toyota", 2022);
        c1.show();
        c2.show();
    }
}
```

Constructor Overloading

You can **overload constructors** by having multiple constructors with **different parameter lists**.

Example:

```
class Box {  
    int length, width;  
  
    Box() {  
        length = width = 1;  
    }  
  
    Box(int l) {  
        length = width = l;  
    }  
  
    Box(int l, int w) {  
        length = l;  
        width = w;  
    }  
  
    void display() {  
        System.out.println("Length: " + length + ", Width: " + width);  
    }  
}
```

Important Notes:

Concept	Explanation
Compiler provides default constructor	Only if no constructor is defined
If any constructor is defined	Compiler does not provide default one
Constructor can't be static	Because it's related to object creation
Can't return value	Constructors never return values
Can use this()	To call another constructor in same class
Constructors aren't inherited	But you can call super() for parent

Constructor vs Method

Feature	Constructor	Method
Purpose	Initializes object	Performs actions
Name	Same as class name	Any valid identifier
Return type	No return type	Must have return type
Called by	JVM when object is created	Called manually
Inheritance	Not inherited	Inherited

Summary Table:

Type of Constructor	Parameters	Who defines it?	Purpose
Default	 No	Compiler (if none given)	Create object with default values
Parameterized	 Yes	Programmer	Create object with specific values

Naming Conventions in Java

What are Naming Conventions?

Naming conventions are **standardized rules** used for naming variables, methods, classes, constants, and other identifiers in Java to improve **readability, consistency, and maintainability** of code.

Java follows **camelCase** and **PascalCase**(also known as **UpperCamelCase**) styles depending on the type of identifier.

Why Naming Conventions Are Important

- Improves **code readability**
 - Promotes **best practices**
 - Makes code easier to **maintain** and **collaborate**
 - Helps follow **industry standards**
-

Types of Naming Conventions in Java

Element	Convention Name	Style Example
Class	PascalCase	StudentDetails
Interface	PascalCase	Runnable, Serializable
Method	camelCase	calculateTotal()
Variable	camelCase	studentName
Constant	UPPER_SNAKE_CASE	MAX_VALUE
Package	lowercase (dots)	com.company.project
Enum	PascalCase (name) + UPPER_CASE (constants)	Day, MONDAY

1. Class Naming Convention

- Use **PascalCase** (each word starts with a capital letter).
- Should be a **noun** (because class represents a thing/entity).

```
class StudentDetails {  
}
```

2. Interface Naming Convention

- Also use **PascalCase**.

```
interface Printable {  
}
```

3. Method Naming Convention

- Use **camelCase** (first word lowercase, next words capitalized).

```
void calculateMarks() {  
}
```

4. Variable Naming Convention

- Use **camelCase**.
- Should be **descriptive and meaningful**.
- Avoid single-letter names except in small loops (*i*, *j*).

```
int studentAge;  
String firstName;
```

5. Constant Naming Convention

- Use **UPPERCASE with underscores**.
- Constants are usually marked as **static final**.

```
public static final int MAX_SPEED = 120;
```

6. Package Naming Convention

- Always use **lowercase** to avoid conflicts with class names.
- Often starts with **reversed domain name**.

```
package com.google.maps;
```

7. Enum Naming Convention

- Enum **type** → PascalCase
- Enum **constants** → UPPERCASE

```
enum Day {  
    MONDAY, TUESDAY, WEDNESDAY  
}
```

Quick Examples Summary:

```
// Class  
class StudentRecord { }  
  
// Interface  
interface Drivable { }  
  
// Method  
void getDetails() { }  
  
// Variable  
int studentRollNo;  
  
// Constant  
public static final double PI = 3.14;  
  
// Package  
package com.mycompany.myapp;  
  
// Enum  
enum Status { ACTIVE, INACTIVE, DELETED; }
```

Common Mistakes to Avoid

Bad Practice	Good Practice
Studentdetails	StudentDetails
CalculateTotal()	calculateTotal()
MAXSPEED	MAX_SPEED
com.Company.Project	com.company.project

Anonymous Object in Java

✓ What is an Anonymous Object?

An **anonymous object** in Java is an object that is **created without assigning it a reference variable**.

It is used **once**, typically for **method calling** or **passing as an argument**, and then it's discarded.

✓ Syntax:

```
new ClassName().methodName();
```

Example:

```
class Demo {
    void show() {
        System.out.println("Anonymous object example");
    }
}

class Main {
    public static void main(String[] args) {
        new Demo().show(); // anonymous object
    }
}
```

The object of **Demo** class is created anonymously and used immediately to call the **show()** method.

NOTE: We can't use the new **Demo()** object again as there is no reference variable. If we do **new Demo()** again then another object will get created.

Inheritance in Java

1. What is Inheritance?

Inheritance is a mechanism in Java by which one class **acquires the properties and behaviors (fields and methods)** of another class.

- The **class being inherited from** is called the **parent class** (or **superclass**).
- The **class that inherits** is called the **child class** (or **subclass**).

Syntax of Inheritance in Java:

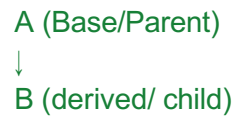
```
class Parent {  
    // fields and methods  
}  
  
class Child extends Parent {  
    // additional fields and methods  
}
```

✓ The **extends** keyword is used to inherit a class.

Types of Inheritance in Java

Type	Description	Supported in Java
Single Inheritance	One class inherits from another class	✓ Yes
Multilevel Inheritance	A class inherits from a class, which in turn inherits another	✓ Yes
Hierarchical Inheritance	Multiple classes inherit from a single parent class	✓ Yes
Multiple Inheritance	A class inherits from more than one class	✗ Not with classes
Hybrid Inheritance	Combination of multiple types of inheritance	✓ Only with interfaces

1. Single Inheritance:



```
class Animal {  
    void eat() { System.out.println("Eating"); }  
}  
  
class Dog extends Animal {  
    void bark() { System.out.println("Barking"); }  
}
```

HERE, one base class and one derived class is there.

2. Multilevel Inheritance



```
class A {  
    void displayA() { System.out.println("A"); }  
}  
  
class B extends A {  
    void displayB() { System.out.println("B"); }  
}  
  
class C extends B {  
    void displayC() { System.out.println("C"); }  
}
```

3. Hierarchical Inheritance



```
class Animal {  
    void sound() { System.out.println("Some sound"); }  
}  
  
class Dog extends Animal {  
    void bark() { System.out.println("Bark"); }  
}  
  
class Cat extends Animal {  
    void meow() { System.out.println("Meow"); }  
}
```

4. Multiple Inheritance (✗ Not Supported via Classes in JAVA)

```
class A {  
    void show() {}  
}  
  
class B {  
    void show() {}  
}  
  
// ✗ Error in Java  
class C extends A, B { // Not allowed  
}
```

Why Java Does NOT Support Multiple Inheritance (with classes)?

✓ Ambiguity Problem:

- If two parent classes have the same method and the child class inherits both, Java cannot decide which method to use.

Example:

```
class A {  
    void show() { System.out.println("A"); }  
}
```

```
class B {  
    void show() { System.out.println("B"); }  
}
```

// class C extends A, B ❌ Not allowed

To avoid such confusion and maintain simplicity, Java does not allow multiple inheritance with classes.

Inheritance Type	Supported via Class	Supported via Interface	Example
Single Inheritance	✅ Yes	✅ Yes	$A \rightarrow B$
Multilevel Inheritance	✅ Yes	✅ Yes	$A \rightarrow B \rightarrow C$
Hierarchical Inheritance	✅ Yes	✅ Yes	$A \rightarrow B, A \rightarrow C$
Multiple Inheritance	❌ No	✅ Yes	Interface A, B $\rightarrow$ C

this() and super() in Java

What are this() and super()?

- Both are **constructor calling statements**.
 - They must be the **first statement** inside a constructor.
-

1. this() – Calls another constructor in the same class

Used for **constructor chaining** within the same class.

Syntax:

```
this(arguments);
```



Example:

```
class Student {
    String name;
    int age;

    Student() {
        this("Default", 18); // calls the 2-arg constructor
    }

    Student(String name, int age) {
        this.name = name;
        this.age = age;
    }

    void display() {
        System.out.println(name + " - " + age);
    }
}

public class Main {
    public static void main(String[] args) {
        Student s = new Student(); // Output: Default - 18
        s.display();
    }
}
```

Rules for **this()**:

- Must be the **first line** in the constructor.
 - Helps **avoid code duplication** by reusing constructor logic.
-

2. **super()** – Calls the parent class constructor

Used in **inheritance** to call the **immediate superclass** constructor.

Syntax:

```
super(arguments);
```

Example:

```
class Animal {  
    Animal(String name) {  
        System.out.println("Animal: " + name);  
    }  
}  
  
class Dog extends Animal {  
    Dog() {  
        super("Dog"); // calls Animal(String)  
        System.out.println("Dog constructor");  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {  
        Dog d = new Dog();  
    }  
}
```

Output:

```
Animal: Dog  
Dog constructor
```

Rules for `super()`:

- Must be the **first line** in the constructor of the subclass.
- If not written explicitly, **Java automatically adds `super()`** (default constructor of parent).
- If parent class has **no default constructor**, you **must call** a parameterized one using `super(...)`.

Comparison Table: `this()` vs `super()`

Feature	<code>this()</code>	<code>super()</code>
Purpose	Calls another constructor in the same class	Calls parent class constructor
Used in	Constructor	Constructor
Must be first statement?	✓ Yes	✓ Yes
Works with inheritance?	✗ No	✓ Yes
Can be used together?	✗ Not in same constructor	Only one can be first line

Summary

- Use `this()` when chaining **constructors in the same class**.
- Use `super()` when initializing **parent class state** during object construction.

OOP: Method Overriding

What is Method Overriding?

Method Overriding occurs when a **subclass** provides a **specific implementation** of a method that is already defined in its **superclass**.

In other words, the **same method signature**, but **different behavior** in child class.

Syntax of Method Overriding

```
class Parent {  
    void show() {  
        System.out.println("Parent's show");  
    }  
}  
  
class Child extends Parent {  
    @Override  
    void show() {  
        System.out.println("Child's show");  
    }  
}
```

Key Rules of Method Overriding

Rule	Description
①	Method must have same name, return type, and parameters as in the parent class
②	Must be in a subclass (i.e., inheritance must exist)
③	Access modifier of overriding method should be same or more accessible (e.g., protected → public is okay)
④	Cannot override final , static , or private methods
⑤	Constructors can't be overridden
⑥	Overridden method can use @Override annotation (recommended)

Example of Method Overriding:

```
class Animal {  
    void sound() {  
        System.out.println("Animal makes a sound");  
    }  
}  
  
class Dog extends Animal {  
    @Override  
    void sound() {  
        System.out.println("Dog barks");  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {  
        Animal a = new Dog(); // Upcasting  
        a.sound();             // Output: Dog barks  
    }  
}
```

This is runtime polymorphism – the method that gets executed is determined at runtime.

Java Packages

What is a Package in Java?

A package is a namespace that organizes a set of related classes and interfaces.

It helps:

- Avoid naming conflicts
- Make classes easier to locate and use
- Provide access control and code modularity

Why Use Packages?

Benefit	Explanation
Namespace Management	Prevents class name collisions (e.g., <code>java.util.Date</code> vs <code>sql.Date</code>)
Code Reusability	Group related classes logically (e.g., utility classes)
Access Control	Use <code>protected/default</code> modifiers to restrict visibility
Code Organization	Maintain clean directory structure

Types of Packages

Type	Description	Example
Built-in	Provided by Java SDK	<code>java.lang</code> , <code>java.util</code> , <code>java.io</code>
User-defined	Created by programmers	<code>com.myapp.utils</code> , <code>school.student</code>

Common Built-in Packages

Package	Description
<code>java.lang</code>	Core classes (String, Math, Object) – auto-imported
<code>java.util</code>	Collections, Date, Scanner, etc
<code>java.io</code>	Input/output (File, Streams)
<code>java.sql</code>	Database connectivity (JDBC)

Creating a Package

✓ Steps:

1. Use **package** keyword as the first line in the Java file.
2. Save the file in a folder with the same name as the package.

// File: MyClass.java
package mypackage;

```
public class MyClass {  
    public void display() {  
        System.out.println("Hello from MyClass");  
    }  
}
```

Directory structure:

```
project/  
├── mypackage/  
│   └── MyClass.java
```

Using a Class from Another Package

Using **import** keyword

```
import mypackage.MyClass; // Single class
```

```
public class Test {  
    public static void main(String[] args) {  
        MyClass obj = new MyClass();  
        obj.display();  
    }  
}
```

Using fully qualified name (without **import**)

```
public class Test {  
    public static void main(String[] args) {  
        mypackage.MyClass obj = new mypackage.MyClass();  
        obj.display();  
    }  
}
```

Access Modifiers

What are Access Modifiers?

Access Modifiers in Java are keywords used to **control the visibility** (accessibility) of **classes, constructors, methods, and variables**.

They define **where** and **how** a particular member can be accessed in other parts of the program.

Types of Access Modifiers

Java provides **four** main access levels:

Modifier	Keyword
Public	<code>public</code>
Protected	<code>protected</code>
Default	<i>No keyword</i> (also called "package-private")
Private	<code>private</code>

1. `public` Modifier

- Most open modifier.
- Accessible **from anywhere**, even outside the package.

```
public class Demo {  
    public int x = 10;  
    public void display() {  
        System.out.println("Public Method");  
    }  
}
```

Can be accessed from any class using object or static reference.

2. `private` Modifier

- Most restricted access.
- Members are **accessible only within the same class**.
- Cannot be inherited.

```

public class Demo {
    private int data = 100;

    private void show() {
        System.out.println("Private data: " + data);
    }
}

```

✗ Cannot be accessed from any other class.

3. **Default (No Modifier)**

- Also called **package-private**.
- Accessible **only within the same package**.
- Not accessible from subclasses in **other packages**.

```

class Demo {
    int number = 20; // default access
    void print() {
        System.out.println("Default access");
    }
}

```

✗ Can't be accessed from a class in a different package.

4. **protected Modifier**

- Accessible within the **same package**.
- Accessible in **subclasses (even in different packages)** via inheritance.

```

class Animal {
    protected void sound() {
        System.out.println("Animal sound");
    }
}

class Dog extends Animal {
    void bark() {
        sound(); // ✅ Inherited
    }
}

```

Cannot be accessed using an object outside the package directly, only through inheritance.

Access Modifiers with access:

Access Level	private	protected	public	default (no modifier)
Same class	✓ Yes	✓ Yes	✓ Yes	✓ Yes
Same package subclass	✗ No	✓ Yes	✓ Yes	✓ Yes
Same package non-subclass	✗ No	✓ Yes	✓ Yes	✓ Yes
Different package subclass	✗ No	✓ Yes	✓ Yes	✗ No
Different package non-subclass	✗ No	✗ No	✓ Yes	✗ No

Polymorphism

What is Polymorphism?

Polymorphism means "many forms".

In Java, it allows objects to behave differently based on the **context** in which they are used.

Types of Polymorphism in Java

Type	Also Called	When It Happens
Compile-time	Method Overloading	During compilation
Run-time	Method Overriding	At runtime

1. Compile-time Polymorphism (Method Overloading)

Same method name, different parameters (type, number, or order).

Example:

```
class Calculator {  
    int add(int a, int b) {  
        return a + b;  
    }  
  
    double add(double a, double b) {  
        return a + b;  
    }  
  
    int add(int a, int b, int c) {  
        return a + b + c;  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {  
        Calculator c = new Calculator();  
        System.out.println(c.add(2, 3));           // 5  
        System.out.println(c.add(2.5, 3.5));       // 6.0  
        System.out.println(c.add(1, 2, 3));        // 6  
    }  
}
```

Key Points:

- Done by **method overloading**.
- Handled by the compiler.
- Improves code readability and flexibility.

2. Run-time Polymorphism (Method Overriding)

Subclass overrides a method of the superclass using the **same method signature**.

Example:

```
class Animal {  
    void sound() {  
        System.out.println("Animal makes a sound");  
    }  
}  
  
class Dog extends Animal {  
    @Override  
    void sound() {  
        System.out.println("Dog barks");  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {  
        Animal a = new Dog(); // upcasting  
        a.sound();             // Output: Dog barks  
    }  
}
```

Key Points:

- Uses **inheritance** and **method overriding**.
- Resolved at **runtime**.
- Enables **dynamic behavior**.

Dynamic Method Dispatch

Dynamic Method Dispatch is the **process of resolving overridden method calls** at runtime using **base class references** pointing to **subclass objects**.

Example:

```
class Shape {  
    void draw() {  
        System.out.println("Drawing a shape");  
    }  
}  
  
class Circle extends Shape {  
    @Override  
    void draw() {  
        System.out.println("Drawing a circle");  
    }  
}  
  
class Square extends Shape {  
    @Override  
    void draw() {  
        System.out.println("Drawing a square");  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {  
        Shape s;  
  
        s = new Circle(); // Shape reference to Circle object  
        s.draw();          // Output: Drawing a circle  
  
        s = new Square(); // Shape reference to Square object  
        s.draw();          // Output: Drawing a square  
    }  
}
```

Difference Between Compile-time and Run-time Polymorphism:

Feature	Compile-time Polymorphism	Run-time Polymorphism
Also Called	Method Overloading	Method Overriding
Resolution Time	Compile-time	Runtime
Involves	Same class	Inheritance
Flexibility	Less flexible	More flexible (dynamic)
Uses	Parameter type/number/order	Same method signature

Summary

- **Compile-time Polymorphism** → Method Overloading
- **Run-time Polymorphism** → Method Overriding
- **Dynamic Method Dispatch** → Calls overridden method via superclass reference

final Keyword in Java

What is final in Java?

final is a **non-access modifier** used to **restrict modification**.

It can be applied to:

- **Variables**
- **Methods**
- **Classes**

Each use of **final** prevents change in different ways.

1. final Variable (Constant)

A **final** variable can **only be assigned once**.
Once set, its value **cannot be changed**.

Example:

```
class Example {  
    final int MAX_USERS = 100;  
  
    void show() {  
        // MAX_USERS = 200; ❌ Error: cannot assign a value to final variable  
        System.out.println(MAX_USERS);  
    }  
}
```

Key Points:

- Must be initialized **when declared** or in the **constructor**.
- Commonly used to define **constants**.
- Naming convention: Use **UPPER_CASE** for final constants.

Initialization in Constructor:

```
class Student {  
    final int id;  
  
    Student(int id) {  
        this.id = id; // allowed once  
    }  
}
```

2. final Method

A **final** method **cannot be overridden** by subclasses.

Example:

```
class Animal {  
    final void sound() {  
        System.out.println("Animal sound");  
    }  
}  
  
class Dog extends Animal {  
    // void sound() {} ❌ Error: Cannot override final method  
}
```

Key Points:

- Prevents method modification through inheritance.
- Often used when behavior should be **unchangeable** for security or design reasons.

3. final Class

A **final** class **cannot be extended (inherited)**.

Example:

```
final class Vehicle {  
    void run() {  
        System.out.println("Vehicle running");  
    }  
}
```

```
// class Car extends Vehicle {} ❌ Error: Cannot inherit from final class
```

Key Points:

- Used when class implementation must be **completely locked**.
- Common in **immutable classes** like `java.lang.String`.

Summary Table

Keyword Use	Restricts...	Can Be Used With	Example
final variable	Value assignment (only once)	variables	<code>final int x = 10;</code>
final method	Method overriding in subclass	methods	<code>final void display()</code>
final class	Inheritance / subclassing	classes	<code>final class Demo {}</code>

Upcasting and Downcasting + abstract keyword+ Inner classes

Upcasting and Downcasting

1. Upcasting

Definition:

Upcasting is the process of converting a subclass type object to a superclass type. It is done implicitly by the compiler.

Syntax:

Superclass obj = new Subclass();

Example:

```
class Animal {
    void sound() {
        System.out.println("Animal makes sound");
    }
}

class Dog extends Animal {
    void bark() {
        System.out.println("Dog barks");
    }
}

public class Main {
    public static void main(String[] args) {
        Animal a = new Dog(); // Upcasting
        a.sound();             // Accessible
        // a.bark();           // Not accessible, compile-time error
    }
}
```

Key Points:

- Upcasting is implicit and safe.
- Only members of the superclass are accessible using the superclass reference.
- Useful in achieving runtime polymorphism.
- Allows treating objects of different subclasses in a uniform way.

2. Downcasting

Definition:

Downcasting is the process of converting a superclass reference back into a subclass reference. It must be done explicitly.

Syntax:

Subclass obj = (Subclass) superclassReference;

Example:

```
class Animal {
    void sound() {
        System.out.println("Animal makes sound");
    }
}

class Dog extends Animal {
    void bark() {
        System.out.println("Dog barks");
    }
}

public class Main {
    public static void main(String[] args) {
        Animal a = new Dog();    // Upcasting
        Dog d = (Dog) a;         // Downcasting
        d.bark();                // Now accessible
    }
}
```

3. Difference Between Upcasting and Downcasting

Aspect	Upcasting	Downcasting
Type	Subclass to Superclass	Superclass to Subclass
Cast Required	No (implicit)	Yes (explicit)
Safety	Safe	Risky if not checked
Access	Only superclass members	Full access to subclass members
Common Usage	Polymorphism, abstraction	When subclass-specific features are needed

Wrapper Classes

Definition:

Wrapper classes in Java are **object representations** of primitive data types. They "wrap" primitive types into objects so they can be used in places where objects are required (e.g., in collections like `ArrayList`).

Java's Primitive Types and Their Wrapper Classes:

Primitive Type	Wrapper Class
byte	Byte
short	Short
int	Integer
long	Long
float	Float
double	Double
char	Character
boolean	Boolean

2. Why Use Wrapper Classes?

- To store primitive values in Java **Collections** (like `ArrayList`, `HashMap`) which only work with objects.
- To use **utility methods** for type conversions and parsing (e.g., `Integer.parseInt()`).
- To support **autoboxing** and **unboxing** features introduced in Java 5.
- Useful when dealing with **null** values (primitive types can't be null).

3. Creating Wrapper Objects

Using Constructors (Not Recommended - Deprecated since Java 9):

```
Integer i = new Integer(5); // Deprecated
```

Using **valueOf()** (Preferred way):

```
Integer i = Integer.valueOf(5);  
Double d = Double.valueOf("3.14");
```

=>Autoboxing (Java automatically converts primitives to objects):

```
int a = 10;  
Integer i = a; // This is called as Autoboxing
```

4. Unboxing

Definition:

Unboxing is the reverse process of autoboxing. Java automatically converts wrapper class objects into their corresponding primitive types.

```
Integer i = Integer.valueOf(20);  
int a = i; // This is called as Unboxing
```

5. Useful Methods in Wrapper Classes

Common methods (using **Integer** as example):

```
Integer i = Integer.valueOf("123");  
  
int a = i.intValue(); // returns 123 (unboxing)  
int b = Integer.parseInt("456"); // parses String to int  
String s = Integer.toString(789); // converts int to String  
Integer max = Integer.max(10, 20); // returns 20
```

parse<Type>() vs **valueOf()**:

Method	Returns	Example
<code>parseInt("5")</code>	primitive int	<code>int a = Integer.parseInt("5");</code>
<code>valueOf("5")</code>	object Integer	<code>Integer i = Integer.valueOf("5");</code>

6. Comparison and Equality

Using == vs .equals():

```
Integer a = 128;
```

```
Integer b = 128;
```

```
System.out.println(a == b);    // false (different objects)
```

```
System.out.println(a.equals(b)); // true (same value)
```

Note: Values between -128 to 127 are cached in Java, so:

```
Integer x = 100;
```

```
Integer y = 100;
```

```
System.out.println(x == y); // true
```

7. Summary

Feature	Description
Purpose	To wrap primitives into objects
Main Use Cases	Collections, type conversion, nullability
Introduced In	Java 1.1 (Autoboxing in Java 5)
Immutability	All wrapper classes are immutable
Memory Caching	Values from -128 to 127 are cached

8. Practice Example

```
public class WrapperDemo {  
    public static void main(String[] args) {  
        int a = 10;  
        Integer obj = a; // autoboxing  
  
        int b = obj;    // unboxing  
  
        System.out.println("Integer object: " + obj);  
        System.out.println("Primitive int: " + b);  
    }  
}
```

abstract Keyword

1. What is the abstract Keyword?

The **abstract** keyword in Java is used for:

- **Declaring a class as abstract** (which cannot be instantiated).
- **Declaring a method as abstract** (which has no implementation in the current class and must be implemented by subclasses).

It is part of Java's **abstraction mechanism** — one of the four pillars of Object-Oriented Programming (OOP).

2. Abstract Class

Definition:

An **abstract class** is a class that is declared using the **abstract** keyword. It **cannot be instantiated directly**.

```
abstract class Shape {  
    abstract void draw(); // abstract method  
    void display() {  
        System.out.println("Displaying shape");  
    }  
}
```

Key Features:

- Can contain **abstract methods** (without a body).
 - Can contain **concrete (non-abstract) methods**.
 - Cannot be instantiated using **new**.
 - Can have **constructors, fields, static methods**, etc.
 - Acts as a **base class** for subclasses to extend.
-

3. Abstract Method

Definition:

An **abstract method** is a method **declared without implementation**. It must be implemented in the subclass.

```
abstract void draw();
```

Characteristics:

- No method body (i.e., ends with a semicolon).
 - Must be declared **inside an abstract class**.
 - Must be overridden by the first **concrete subclass**.
-

4. Example: Abstract Class and Method

```
abstract class Animal {
    abstract void sound();    // abstract method
    void sleep() {
        System.out.println("Sleeping...");
    }
}

class Dog extends Animal {
    void sound() {
        System.out.println("Dog barks");
    }
}

public class Main {
    public static void main(String[] args) {
        Animal a = new Dog(); // Polymorphism
        a.sound();             // Dog barks
        a.sleep();             // Sleeping...
    }
}
```

5. Why Use Abstract Classes?

- To provide a **common base** for all subclasses.
- To **enforce** a contract for subclasses to implement specific methods.
- When you want to provide **partial implementation** in the base class.

6. Rules of Abstract Classes and Methods

Rule	Explanation
Abstract class cannot be instantiated	<code>new</code> keyword cannot create objects of an abstract class
Abstract class can contain constructors	These are called when subclass objects are created
Abstract class can have both abstract and concrete methods	Allows partial implementation
If a class contains even one abstract method , it must be abstract	Otherwise it's a compile-time error
A subclass must override all abstract methods of its superclass	Or it must be declared abstract itself
Abstract class can extend another abstract class	And may or may not implement all abstract methods
Abstract class can have static methods and fields	But static methods cannot be abstract
Interfaces are not abstract classes , but support similar behavior	Since Java 8+, interfaces can have default and static methods too

7. Common Mistakes

Mistake	Problem
Declaring abstract method in non-abstract class	Compile-time error
Instantiating abstract class directly	Compile-time error
Not implementing all abstract methods in subclass	Compile-time error unless subclass is also abstract

8. Abstract Class with Constructor Example

```
abstract class Vehicle {
    Vehicle() {
        System.out.println("Vehicle constructor called");
    }

    abstract void start();
}

class Car extends Vehicle {
    Car() {
        System.out.println("Car constructor called");
    }

    void start() {
        System.out.println("Car started");
    }
}
```

Output:

Vehicle constructor called
Car constructor called
Car started

9. Summary

Concept	Description
<code>abstract</code> keyword	Used to declare abstract classes and methods
Abstract class	Cannot be instantiated, can have both abstract and concrete methods
Abstract method	Declared without a body, must be implemented in subclass
Usage	To achieve abstraction and enforce subclass behavior

Inner Class

Definition:

An **Inner Class** is a class defined **within another class** in Java. It is associated with an instance of the enclosing class and can access its private members.

Types of Inner Classes:

1. **Member Inner Class** (non-static)
 2. **Static Nested Class**
 3. **Local Inner Class** (inside a method)
 4. **Anonymous Inner Class**
-

1.1 Member Inner Class

- Declared **inside a class**, **outside** any method or block.
- Has access to all members (including `private`) of the outer class.
- Requires an instance of the outer class to be instantiated.

```
class Outer {
    int x = 10;

    class Inner {
        void show() {
            System.out.println("x = " + x);
        }
    }

    public static void main(String[] args) {
        Outer outer = new Outer();
        Outer.Inner inner = outer.new Inner();
        inner.show();
    }
}
```

1.2 Static Nested Class

- Declared using `static` keyword inside a class.
- Cannot access **non-static** members of the outer class directly.
- Can be instantiated without an instance of the outer class.

```
class Outer {  
    static int x = 100;  
  
    static class Nested {  
        void display() {  
            System.out.println("x = " + x);  
        }  
    }  
  
    public static void main(String[] args) {  
        Outer.Nested obj = new Outer.Nested();  
        obj.display();  
    }  
}
```

2. Anonymous Inner Class

Definition:

An **Anonymous Inner Class** is a **local inner class** without a name. It is used to create a **one-time subclass** or **implement an interface** on the fly, usually when passing to a method.

Use Case:

- When a class is needed only once.
- When implementing an interface or abstract class without creating a separate named class.

Syntax:

```
InterfaceOrAbstract obj = new InterfaceOrAbstract() {  
    // implementation of methods  
};
```

Example with Abstract Class:

```
abstract class Animal {  
    abstract void makeSound();  
}  
  
public class Test {  
    public static void main(String[] args) {  
        Animal dog = new Animal() {  
            void makeSound() {  
                System.out.println("Bark!");  
            }  
        };  
        dog.makeSound();  
    }  
}
```

3. Abstract Method and Inner Class

Abstract Method Recap:

- A method declared without implementation using the `abstract` keyword.
- Must be overridden in a subclass.

Using Inner Class to Implement Abstract Method:

An **inner class (often anonymous)** can be used to provide the implementation of an abstract method.

Example:

```
abstract class Shape {
    abstract void draw();
}

class Painter {
    void paint() {
        Shape circle = new Shape() {
            void draw() {
                System.out.println("Drawing Circle");
            }
        };
        circle.draw();
    }

    public static void main(String[] args) {
        new Painter().paint();
    }
}
```

3. Abstract Method and Inner Class

Abstract Method Recap:

- A method declared without implementation using the `abstract` keyword.
- Must be overridden in a subclass.

Using Inner Class to Implement Abstract Method:

An **inner class (often anonymous)** can be used to provide the implementation of an abstract method.

Example:

```
abstract class Shape {  
    abstract void draw();  
}  
  
class Painter {  
    void paint() {  
        Shape circle = new Shape() {  
            void draw() {  
                System.out.println("Drawing Circle");  
            }  
        };  
        circle.draw();  
    }  
  
    public static void main(String[] args) {  
        new Painter().paint();  
    }  
}
```

Here:

- `Shape` is abstract.
- Anonymous inner class overrides `draw()` method.
- No need to create a separate subclass.

4. Key Differences

Feature	Inner Class	Anonymous Inner Class
Class Name	Has a name	No name
Reusability	Can be reused	Used once
Constructor	Can have constructors	Cannot define constructors
Use Case	Logical grouping	Ad-hoc subclassing or implementing
Access to Outer Class	Has access	Has access

Summary Points

- **Inner classes** are useful for logically grouping classes and improving encapsulation.
- **Anonymous inner classes** provide quick, one-time implementations of abstract classes or interfaces.
- Inner classes have access to **private members** of the outer class.
- Anonymous classes **cannot define constructors** or **static members**.
- When using **abstract methods**, anonymous inner classes often provide an elegant way to override them **without subclassing**.

CodeWithNishchal

Interfaces in JAVA

1. Definition

An **interface** in Java is a **reference type**, similar to a class, that can contain:

- Only Abstract methods (Even if you don't mention, methods are by default **public abstract**)
- Constants (by default **public static final**)

An interface defines a **contract** that implementing classes must fulfill.

Syntax:

```
interface Vehicle {  
    void start(); // implicitly public and abstract  
}
```

2. Why Do We Need Interfaces in Java?

Key Reasons:

1. Abstraction

- Interfaces help achieve 100% abstraction.
- No implementation logic is forced into the interface itself.

2. Multiple Inheritance

- Java does **not support multiple inheritance with classes** (to avoid the diamond problem).
- But a class **can implement multiple interfaces**, solving this limitation.

3. Polymorphism

- Objects can be referred to using interface types, enabling flexibility and polymorphism.
-

3. Implementing an Interface

```
interface Animal {  
    void sound();  
}  
  
class Dog implements Animal {  
    public void sound() {  
        System.out.println("Bark");  
    }  
}
```

- The class must **implement all abstract methods** of the interface.
- Use the **implements** keyword.
- The methods must be marked **public** (or have at least public access).

4. Implementing Multiple Interfaces

Java allows a class to implement **more than one interface** — enabling **multiple inheritance of type**.

```
interface Printable {  
    void print();  
}  
  
interface Showable {  
    void show();  
}  
  
class A implements Printable, Showable {  
    public void print() {  
        System.out.println("Printing");  
    }  
  
    public void show() {  
        System.out.println("Showing");  
    }  
}
```

Why this is allowed:

- Interfaces **only declare methods**, they do **not provide implementation (unless default)**.
- So, there is **no conflict or ambiguity** like in multiple class inheritance.

5. Interface vs Class Inheritance

Feature	Class Inheritance (extends)	Interface (implements)
Type	Concrete or abstract classes	Abstract reference types
Multiple Inheritance	Not allowed	Allowed
Constructors	Inherited	Not applicable
Fields	Can have instance fields	Only public static final constants
Methods	Can have full implementation	Only abstract, default, static
Access Modifiers	Can be any	Methods are public by default

6. Multiple Inheritance Using Interfaces

Problem with Class Inheritance:

```
class A { void msg() {} }  
class B { void msg() {} }  
class C extends A, B {} // ❌ Not allowed in Java
```

We can solve this using the interface.

Solution using interface:

```

interface A {
    void msg();
}

interface B {
    void msg();
}

class C implements A, B {
    public void msg() {
        System.out.println("Hello from C");
    }
}

```

This works because:

- There is no **method body** in the interfaces (if not using default methods), so no ambiguity.
- The class provides its **own implementation**.

7. Interface Inheritance

An interface can **extend** one or more interfaces:

```

interface A {
    void a();
}

interface B {
    void b();
}

interface C extends A, B {
    void c();
}

```

A class implementing **C** must implement methods from **A**, **B**, and **C**.

8. Important Rules

Rule	Explanation
All interface methods are implicitly abstract (Java 7-)	No need to use abstract keyword
Interface variables are public static final	Constants only
A class can implement multiple interfaces	Multiple inheritance
Interfaces can extend multiple other interfaces	Interface inheritance
Classes must implement all abstract methods	Or be declared abstract

9. Quick Summary Table

Keyword	Purpose
interface	Define an abstract type (no instances)
implements	Class implements interface(s)
extends	Class extends another class or interface extends interface

10. Real-life Analogy

Think of an interface like a **remote control**:

- It declares buttons (methods) like **power()**, **volumeUp()**.
- Any device (TV, AC, etc.) that implements the remote interface must provide **its own behavior** for those buttons.

12. Interview Tip

"Why does Java allow multiple interfaces but not multiple class inheritance?"

✓ Because interfaces do **not bring implementation conflicts**, while multiple classes can have conflicting method bodies (known as the *diamond problem*). Java avoids this by allowing only **multiple interfaces**, not multiple class inheritance.

Interfaces & Functional Programming

Types of Interfaces in Java

There are **3 major types**:

Type of Interface	Methods Count	Purpose	Example Use Cases
Normal Interface	Multiple abstract methods	Multiple related behaviors	Shapes, Vehicle features
Functional Interface	Only one abstract method	Functional Programming, Lambdas	Runnable, Callable, Predicate
Marker Interface	No methods	Provide metadata to JVM	Serializable, Cloneable

1. Normal Interface

A regular interface having **more than one abstract method**.

- Supports Multiple inheritance
- All methods are **public & abstract** by default
- Variables are **public static final** (constants)

```
interface Animal {
    void eat();    // abstract method
    void sleep();  // abstract method
}

class Dog implements Animal {
    @Override
    public void eat() {
        System.out.println("Dog eats food");
    }
    @Override
    public void sleep() {
        System.out.println("Dog sleeps at night");
    }
}

public class Main {
    public static void main(String[] args) {
        Dog d = new Dog();
        d.eat();
        d.sleep();
    }
}
```

2. Functional Interface (Java 8 Feature)

Interface with **exactly ONE** abstract method.

Also called **SAM Interface** → *Single Abstract Method*

✓ Why important?

- Enables **Lambda Expressions**
- Supports **Functional Programming** style in Java

Rules

Rule	Meaning
Must have one abstract method	Otherwise not functional
Can have default and static methods	They don't count as abstract methods
Annotated using <code>@FunctionalInterface</code>	Optional but recommended

Example

```
@FunctionalInterface
interface Sayable {
    void say(String msg); // Single abstract method

    default void greet() {
        System.out.println("Hello default method!");
    }
}

public class Main {
    public static void main(String[] args) {
        Sayable obj = (msg) -> System.out.println(msg); // Lambda Expression
        obj.say("Java Functional Interface");
        obj.greet();
    }
}
```

✓ Built-in Functional Interfaces (Java 8)

Interface	Purpose	Abstract Method
Runnable	Multi-threading task	run ()
Callable<T>	Task returning value	call ()
Comparator<T>	Sorting logic	compare ()
Predicate<T>	Returns boolean condition	test ()
Function<T, R>	Converts one value to another	apply ()
Consumer<T>	Performs action, no return	accept ()
Supplier<T>	Supplies value	get ()

3. Marker Interface

Has **zero** abstract methods They tell JVM/Compiler **extra information** about a class

Marker Interface	Meaning
Serializable	Class can be serialized
Cloneable	Object cloning allowed
Remote	Allows remote communication

```
class Student implements java.io.Serializable {  
    int id;  
    String name;  
  
    Student(int id, String name) {  
        this.id = id;  
        this.name = name;  
    }  
}
```

JVM understands objects of `Student` **can be serialized** even though no method is implemented.

Functional Programming in Java 8

Before Java 8 → Java was **fully OOP**

After Java 8 → Java supports **Functional Programming style** because of:

- ✓ Lambda Expressions
 - ✓ Functional Interfaces
 - ✓ Stream API
 - ✓ Method References
-

Lambda Expressions

Short way to implement **functional interface** method

Syntax:

```
(parameters) -> expression  
or  
(parameters) -> { multiple statements }
```

Example without Lambda

```
Runnable r = new Runnable() {  
    @Override  
    public void run() {  
        System.out.println("Thread is running");  
    }  
};
```

Same Example using Lambda 😎

```
Runnable r = () -> System.out.println("Thread is running");
```

Cleaner + Less Code + Readable

More Lambda Examples

1) With Single Parameter

```
Sayable say = msg -> System.out.println(msg);  
say.say("Hello Lambda!");
```

2 With Multiple Parameters

```
interface MathOps {  
    int add(int a, int b);  
}
```

```
MathOps obj = (a, b) -> a + b;  
System.out.println(obj.add(5, 10));
```

3 Multiple Statements

```
MathOps sub = (a, b) -> {  
    System.out.println("Subtracting");  
    return a - b;  
};  
System.out.println(sub.add(20, 5));
```

When to use Lambdas?

Problem	Use
Single abstract method	Lambda best fit
Collections sorting/filtering	Lambdas + Streams
Background thread	Lambda + Runnable

Interface Type Comparison Table

Feature	Normal Interface	Functional Interface	Marker Interface
No. of abstract methods	2 or more	Exactly 1	0
Used for	General purpose	Lambda / Streams	Metadata
Introduced in	Java 1.0	Java 8	Java 1.0
Can have default/static methods?	Yes	Yes	No methods at all

Enums

1. Definition

An **Enum** (short for enumeration) is a **special data type** in Java used to define a **fixed set of constants**.

```
enum Direction {  
    NORTH, SOUTH, EAST, WEST;  
}
```

Key Features:

- Enum is **type-safe** – meaning you can't assign any other value except those defined. Every enum constant is **public static final**.
- Java Enums are **implicitly a subclass of `java.lang.Enum`**.
- Enums **cannot extend another class**, but they can **implement interfaces**.

2. Why Use Enums?

Feature	Benefit
Fixed set of constants	Prevents illegal values
Can have fields, methods, constructors	Like a class, but restricted
Integrates well with <code>switch</code> , collections, etc.	Easy control flow and mapping

3. Basic Enum Example

```
enum Day {  
    MONDAY, TUESDAY, WEDNESDAY, THURSDAY, FRIDAY, SATURDAY, SUNDAY  
}  
  
public class TestEnum {  
    public static void main(String[] args) {  
        Day today = Day.MONDAY;  
        System.out.println("Today is " + today);  
    }  
}
```

Output: Today is MONDAY

4. Enum with if-else

```
if (today == Day.MONDAY) {  
    System.out.println("Start of the workweek!");  
} else if (today == Day.FRIDAY) {  
    System.out.println("Weekend is near!");  
}
```

5. Enum with switch-case

```
switch (today) {  
    case MONDAY:  
        System.out.println("Back to work!");  
        break;  
    case SATURDAY:  
    case SUNDAY:  
        System.out.println("It's weekend!");  
        break;  
    default:  
        System.out.println("Regular weekday.");  
}
```

6. Enum with Fields, Constructor, and Methods

You can treat an enum **almost like a class**

We can add fields, constructors (private only) and methods as well.

Example:

```
enum Season {  
    WINTER("Cold"),  
    SPRING("Warm"),  
    SUMMER("Hot"),  
    AUTUMN("Cool");  
  
    private String description;  
  
    // Constructor  
    private Season(String desc) {  
        this.description = desc;  
    }  
  
    // Method  
    public String getDescription() {  
        return description;  
    }  
}
```

Usage:

```
public class TestEnum {  
    public static void main(String[] args) {  
        for (Season s : Season.values()) {  
            System.out.println(s + " → " + s.getDescription());  
        }  
    }  
}
```

Output:

WINTER → Cold
SPRING → Warm
SUMMER → Hot
AUTUMN → Cool

7. Important Enum Methods

Method	Description
<code>values()</code>	Returns an array of all enum constants
<code>valueOf("STRING")</code>	Returns the enum constant matching the string
<code>ordinal()</code>	Returns the position (index) of constant (starting from 0)

Example:

```
Season s = Season.valueOf("SUMMER"); // SUMMER
int index = s.ordinal();             // 2
```

8. Enum in Switch Example: Full Code

```
enum TrafficLight {
    RED, GREEN, YELLOW;
}

public class TestSwitch {
    public static void main(String[] args) {
        TrafficLight signal = TrafficLight.GREEN;

        switch (signal) {
            case RED:
                System.out.println("Stop!");
                break;
            case GREEN:
                System.out.println("Go!");
                break;
            case YELLOW:
                System.out.println("Slow down!");
                break;
        }
    }
}
```

Annotations

What Are Annotations in Java?

Annotations are a form of metadata in Java. They provide data about a program but are not part of the program itself. They have no direct effect on the operation of the code they annotate.

Introduced in **Java 5**, annotations are widely used for:

- Compiler instructions
 - Compile-time and runtime processing
 - Configuration (e.g., Spring, JPA)
-

Syntax of Annotations

```
@AnnotationName(parameter1 = value1, parameter2 = value2)
```

Examples:

```
@Override  
public String toString() {  
    return "Example";  
}
```

Built-in Annotations in Java

1. **@Override**

Indicates that a method is intended to override a method in a superclass.

2. **@Deprecated**

Marks a method or class as outdated.

3. @SuppressWarnings

Tells the compiler to ignore specific warnings.

```
@SuppressWarnings("unchecked")
```

4. @SafeVarargs

Ensures that a method with varargs doesn't perform unsafe operations on them.

5. @FunctionalInterface

Indicates that an interface is intended to be a functional interface (one abstract method).

Meta-Annotations

These are annotations **used to annotate other annotations**.

Meta-Annotation	Purpose
@Target	Specifies where the annotation can be applied (e.g., METHOD, FIELD).
@Retention	Indicates how long annotations are retained (SOURCE, CLASS, RUNTIME).
@Documented	Indicates that the annotation should be included in the JavaDoc.
@Inherited	Indicates that the annotation is inherited by subclasses.
@Repeatable	Allows the annotation to be used more than once on the same element.

Example:

```
@Target(ElementType.METHOD)
@Retention(RetentionPolicy.RUNTIME)
public @interface MyAnnotation {
    String value();
}
```

Custom Annotations

1. Define the Annotation

```
import java.lang.annotation.*;

@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.METHOD)
public @interface MyAnnotation {
    String author() default "Unknown";
    int version();
}
```

2. Use the Annotation

```
public class Demo {
    @MyAnnotation(author = "John", version = 1)
    public void show() {
        System.out.println("Custom annotation example");
    }
}
```

3. Access Annotations Using Reflection

```
import java.lang.reflect.Method;

public class TestAnnotation {
    public static void main(String[] args) throws Exception {
        Method method = Demo.class.getMethod("show");
        MyAnnotation annotation = method.getAnnotation(MyAnnotation.class);

        System.out.println("Author: " + annotation.author());
        System.out.println("Version: " + annotation.version());
    }
}
```

Retention Policies

Retention Policy	Description
SOURCE	Discarded by the compiler (used for compile-time only).
CLASS	Stored in the <code>.class</code> file but not available at runtime.
RUNTIME	Available at runtime via reflection.

Target Element Types

ElementType	Usage Example
TYPE	Classes and interfaces
FIELD	Fields (instance variables)
METHOD	Methods
PARAMETER	Method parameters
CONSTRUCTOR	Constructors
LOCAL_VARIABLE	Local variables
ANNOTATION_TYPE	Annotation types
PACKAGE	Packages
TYPE_USE	Type uses (Java 8+)

Exception Handling

Exception Handling in Java is a mechanism used to handle runtime errors, allowing the normal flow of the program to continue. Without exception handling, any error will terminate the program.

Definition of Exception:

An **exception** is an unwanted or unexpected event that occurs during the execution of a program and disrupts its normal flow.

Types of Exceptions

1. Checked Exceptions

- These are exceptions that are checked at **compile-time**.
- The compiler forces the programmer to handle these exceptions.
- Examples: `IOException`, `SQLException`, `ClassNotFoundException`

2. Unchecked Exceptions

- These are exceptions that occur at **runtime**.
- The compiler does not check for these exceptions.
- Examples: `ArithmeticException`, `NullPointerException`, `ArrayIndexOutOfBoundsException`

Why Use Exception Handling?

- To maintain the normal flow of the application even when an error occurs.
- To catch and respond to specific error types.
- To separate error-handling code from regular code, improving readability.

Keywords Used in Exception Handling

Java provides five keywords to handle exceptions:

1. **try** - Defines a block of code to be tested for errors.
2. **catch** - Defines a block of code that handles the exception.
3. **finally** - Defines a block that is always executed, whether an exception is handled or not.
4. **throw** - Used to explicitly throw an exception.
5. **throws** - Declares the exceptions that a method can throw.

try and catch Blocks

Syntax:

```
try {  
    // Code that may throw an exception  
} catch (ExceptionType e) {  
    // Code to handle the exception  
}
```

- The code inside the **try** block is monitored for exceptions.
- If an exception occurs, control is transferred to the appropriate **catch** block.

Example:

```
public class Example {  
    public static void main(String[] args) {  
        try {  
            int result = 10 / 0; // This will cause ArithmeticException  
            System.out.println("Result: " + result);  
        } catch (ArithmeticException e) {  
            System.out.println("Exception caught: " + e.getMessage());  
        }  
  
        System.out.println("Program continues...");  
    }  
}
```

Output:

Exception caught: / by zero
Program continues...

Explanation:

- The division by zero causes an `ArithmeticException`.
- The `catch` block handles the exception and prevents the program from crashing.
- The message `/ by zero` is printed from the exception.
- Execution continues after the catch block.

Multiple catch Blocks

You can have more than one catch block to handle different types of exceptions:

```
try {  
    int[] arr = new int[5];  
    arr[10] = 100; // Causes ArrayIndexOutOfBoundsException  
} catch (ArithmeticException e) {  
    System.out.println("Arithmetic Exception occurred");  
} catch (ArrayIndexOutOfBoundsException e) {  
    System.out.println("Array Index Out of Bounds Exception occurred");  
} catch (Exception e) {  
    System.out.println("Some other exception occurred");  
}
```

- The most specific exceptions should be caught first.
- The general `Exception` should be caught last.

Flow of try-catch

1. The code in the `try` block is executed.
2. If no exception occurs, the `catch` blocks are skipped.
3. If an exception occurs, the rest of the `try` block is skipped.
4. The corresponding `catch` block is executed.

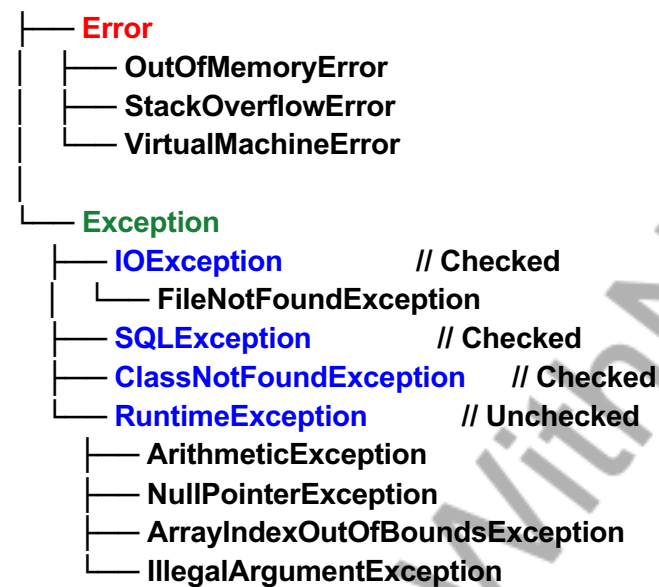
Exception Hierarchy:

1. Throwable

- Root class for all exceptions and errors in Java.
- Has two main direct subclasses:
 - **Error** – Serious system errors.
 - **Exception** – Problems that can be handled.

2. Under Throwable:

Throwable



Summary Table:

Class	Type	Compile-Time Check	Examples
Error	Unchecked	No	OutOfMemoryError, StackOverflowError
Exception	Checked	Yes	IOException, SQLException
RuntimeException	Unchecked	No	NullPointerException, ArithmeticException

Throw

1. The **throw** Keyword in Java

Definition:

The **throw** keyword is used to **explicitly throw an exception** from a method or any block of code.

Syntax:

```
throw new ExceptionType("Error Message");
```

Key Points:

- Used to **manually throw** a single exception.
- It must throw an object of type **Throwable** (either **Exception** or **Error**).
- It is typically used in **custom validation** or **business rules**.

Example 1: Throwing an unchecked exception

```
public class ThrowExample {  
    public static void main(String[] args) {  
        int age = 15;  
  
        if (age < 18) {  
            throw new ArithmeticException("Access denied - You must be 18+.");  
        }  
  
        System.out.println("Access granted.");  
    }  
}
```

Output:

Exception in thread "main" java.lang.ArithmeticException: Access denied - You must be 18+.

Example 2: Throwing a checked exception

```
public class CheckedThrow {  
    public static void main(String[] args) throws Exception {  
        throw new Exception("This is a checked exception");  
    }  
}
```

Note: In this case, we must use `throws` in the method signature or handle the exception using `try-catch`.

Throws

What is `throws`?

The `throws` keyword in Java is used in a **method declaration** to indicate that this method **might throw an exception**. It tells the **caller of the method** that they must handle the exception using `try-catch` or declare it further using `throws`.

Key Points:

- Used only in **method signatures**, not in the body.
- Used mostly with **checked exceptions** (but can be used with unchecked too).
- Helps in **propagating exceptions** up the call stack.

Syntax:

```
returnType methodName() throws ExceptionType {  
    // method body  
}
```

Example 1: Two Methods — Division by Zero with **throws**

This example shows **2 methods**, and one method throws an unchecked exception (**ArithmeticException**) to the calling method using **throws**.

```
public class ThrowsExample1 {  
  
    static void divide() throws ArithmeticException {  
        int a = 10, b = 0;  
        int result = a / b; // This will throw ArithmeticException  
        System.out.println("Result: " + result);  
    }  
  
    public static void main(String[] args) {  
        try {  
            divide(); // calling method that throws exception  
        } catch (ArithmeticException e) {  
            System.out.println("Caught an exception: " + e.getMessage());  
        }  
    }  
}
```

Output:

Caught an exception: / by zero

Explanation:

- **divide()** method declares that it may throw **ArithmeticException**.
- **main()** handles it using **try-catch**.
-

Example 2: Three Methods – Exception Propagation with **throws**

This example shows **3 methods**. The lowest method throws an exception, and it's declared with **throws** all the way up to the **main()** method.

```

public class ThrowsExample2 {

    static void methodA() throws ArithmeticException {
        int x = 100, y = 0;
        int result = x / y; // Throws ArithmeticException
        System.out.println("Result: " + result);
    }

    static void methodB() throws ArithmeticException {
        methodA(); // call method that throws exception
    }

    static void methodC() throws ArithmeticException {
        methodB(); // call method that throws exception
    }

    public static void main(String[] args) {
        try {
            methodC(); // start the call chain
        } catch (ArithmeticException e) {
            System.out.println("Exception handled in main: " + e.getMessage());
        }
    }
}

```

Output:

Exception handled in main: / by zero

Explanation:

- `methodA()` throws `ArithmeticException`.
- `methodB()` and `methodC()` **propagate** the exception using `throws`.
- `main()` catches the exception and handles it.

Summary of How **throws** Works:

- You declare a method may throw an exception using `throws`.
- If the method that calls it **does not handle it**, it must **also declare it with `throws`**.
- This is called **exception propagation**.
- Eventually, some method (often `main()`) must catch and handle the exception.

Custom Exceptions in Java

What is a Custom Exception?

In Java, a **custom exception** is a user-defined exception class that extends either:

- `Exception` (for a **checked exception**) or
- `RuntimeException` (for an **unchecked exception**)

Why Create Custom Exceptions?

- To represent **application-specific error conditions**
- To make your code **easier to understand and debug**
- To implement **business logic violations**, such as "Insufficient Balance" or "Invalid Age"

1. Types of Custom Exceptions

Type	Extends	Checked/Unchecked	Must Handle?
Checked Exception	<code>Exception</code>	Checked	Yes (try-catch or throws)
Unchecked Exception	<code>RuntimeException</code>	Unchecked	No (optional to catch)

2. Syntax to Create a Custom Exception:

a. Checked Exception

```
class MyCheckedException extends Exception {  
    public MyCheckedException(String message) {  
        super(message); // pass message to parent Exception class  
    }  
}
```

b. Unchecked Exception

```
class MyUncheckedException extends RuntimeException {  
    public MyUncheckedException(String message) {  
        super(message);  
    }  
}
```

3. Using a Custom Exception – Example

Example: Custom Checked Exception

```
// Step 1: Create the custom exception
class InvalidAgeException extends Exception {
    public InvalidAgeException(String message) {
        super(message);
    }
}

// Step 2: Use it in code
public class CustomExceptionExample {
    public static void validateAge(int age) throws InvalidAgeException {
        if (age < 18) {
            throw new InvalidAgeException("Age must be 18 or above.");
        } else {
            System.out.println("Valid age for voting.");
        }
    }

    public static void main(String[] args) {
        try {
            validateAge(16);
        } catch (InvalidAgeException e) {
            System.out.println("Caught Exception: " + e.getMessage());
        }
    }
}
```

Output:

Caught Exception: Age must be 18 or above.

4. Key Components of a Custom Exception

Component	Description
Constructor	Used to pass error messages
<code>super(message)</code>	Passes the message to the parent class constructor
Optional Override	You can override methods like <code>toString()</code>

User Input Methods in Java

Java provides multiple ways to take input from users. The two most commonly used are:

1. `Scanner` (introduced in Java 5)
2. `BufferedReader` (older, from Java 1.1)

1. Scanner Class

`Scanner` is a **simple and modern** way to read user input from various sources like Keyboard (standard input), Files and Strings.

Package:

```
import java.util.Scanner;
```

Syntax:

```
Scanner sc = new Scanner(System.in);
```

Common Methods:

Method	Description
<code>next()</code>	Reads a single word (no spaces)
<code>nextLine()</code>	Reads an entire line
<code>nextInt()</code>	Reads an integer
<code>nextDouble()</code>	Reads a double
<code>nextBoolean()</code>	Reads a boolean

Example:

```
import java.util.Scanner;

public class ScannerExample {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter your name: ");
        String name = sc.nextLine();

        System.out.print("Enter your age: ");
        int age = sc.nextInt();

        System.out.println("Hello " + name + ", you are " + age + " years old.");
    }
}
```

2. BufferedReader Class

BufferedReader is a **more traditional and efficient** way to read text from the input stream (keyboard, files, etc.). It reads input **line by line**.

Package:

```
import java.io.BufferedReader;
import java.io.InputStreamReader;
```

Syntax:

```
BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
```

Common Method:

Method	Description
<code>readLine()</code>	Reads a full line as a String

Note: It returns everything as a **String**, so you must convert it manually.

Example:

```

import java.io.BufferedReader;
import java.io.InputStreamReader;
import java.io.IOException;

public class BufferedReaderExample {
    public static void main(String[] args) throws IOException {
        BufferedReader br = new BufferedReader(new InputStreamReader(System.in));

        System.out.print("Enter your name: ");
        String name = br.readLine();

        System.out.print("Enter your age: ");
        int age = Integer.parseInt(br.readLine()); // Manual parsing

        System.out.println("Hello " + name + ", you are " + age + " years old.");
    }
}

```

Internal Working of System.out.println()

When you write:

```
System.out.println("Hello");
```

You're actually using **multiple layers of objects and classes**. Here's what happens behind the scenes:

1. System Class

- **System** is a **final class** in **java.lang** package.
- It **cannot be instantiated**.
- It contains **static fields** and **static methods**.

```

public final class System {

    public static final PrintStream out = ...;

}

```

Key point:

- `System.out` is a **public static final field** of type `PrintStream`.
 - Since it's **static**, you access it directly using `System.out`.
-

2. `out` is a `PrintStream` Object

- `out` is an instance of the `PrintStream` class.
- It is initialized to point to the **standard output stream**, usually the **console**.

Inside `System.java` (simplified version):

```
public static final PrintStream out = new PrintStream(new BufferedOutputStream(new
FileOutputStream(FileDescriptor.out)), true);
```

- This sets `System.out` to write output to the terminal.

3. `PrintStream` Class

- The class `java.io.PrintStream` provides methods to print data to output streams.
- It contains several `println()` methods that accept different data types like `int`, `String`, `char`, etc.

Method Signatures in `PrintStream`:

```
public void println(String x)
```

```
public void println(int x)
```

```
public void println(char x)
```

These methods:

- Convert the data to string (if needed)
- Append a newline `\n`
- Write the string to the output stream (console)

4. Flow of `System.out.println("Hello");`

Let's trace what happens:

Code:

```
System.out.println("Hello");
```

Step-by-step:

1. `System` is a class that contains a static field called `out`.
 2. `out` is a reference to a `PrintStream` object.
 3. `println("Hello")` is a method of `PrintStream`.
 4. The string `"Hello"` is written to the output stream (terminal) with a newline character.
-

5. Summary

Part	Description
<code>System</code>	Final class in <code>java.lang</code> with static fields
<code>System.out</code>	Static <code>PrintStream</code> object connected to the console
<code>println()</code>	Method of <code>PrintStream</code> to print data + newline
Output goes to	Standard output stream (usually the terminal/console)

Finally

1. **finally** Block in Java

The **finally** block is used to **execute important code such as closing resources** (files, database connections, sockets), regardless of whether an exception was handled or not.

Key Characteristics:

- It **always executes**, whether an exception is thrown or not.
- It runs **after try and catch blocks**.
- It is **optional** in exception handling.
- Can be used **with or without a catch block**, but must follow a **try**.

Syntax:

```
try {  
    // risky code  
} catch (Exception e) {  
    // exception handling  
} finally {  
    // cleanup code (always executes)  
}
```

Example:

```
public class FinallyExample {  
    public static void main(String[] args) {  
        try {  
            int a = 10 / 0; // throws ArithmeticException  
        } catch (ArithmeticException e) {  
            System.out.println("Exception caught: " + e.getMessage());  
        } finally {  
            System.out.println("Finally block always executes.");  
        }  
    }  
}
```

Output:

Exception caught: / by zero

Finally block always executes.

Can **finally** block be skipped?

Yes, but **only in special cases**, such as:

- JVM crashes
- `System.exit()` is called inside `try` or `catch`

When to use **finally**:

- Releasing resources (closing files, DB connections)
- Logging and cleanup tasks

try-with-resources

It ensures:

- Resources are **closed automatically** after the try block
- Less error-prone and cleaner code than using `finally`

Syntax:

```
try (ResourceType resource = new ResourceType()) {  
    // use the resource  
} catch (ExceptionType e) {  
    // handle exception  
}
```

Example with FileReader:

```
import java.io.*;  
  
public class TryWithResourcesExample {  
    public static void main(String[] args) {  
        try (BufferedReader br = new BufferedReader(new FileReader("data.txt"))) {  
            String line;  
            while ((line = br.readLine()) != null) {  
                System.out.println(line);  
            }  
        } catch (IOException e) {  
            System.out.println("Error: " + e.getMessage());  
        }  
    }  
}
```

What's Happening?

- `BufferedReader` is declared in the parentheses of the `try` block.
- After execution, Java **automatically calls `br.close()`**, even if an exception occurs.
- No need for a `finally` block to close the resource.

CodeWithNishchal

Unit Testing with JUnit

1. What is Unit Testing?

Unit testing is the process of testing **individual units or components** of a program in isolation to ensure that each part functions correctly.

- Each "unit" is typically a single method or function.
 - Focuses on **correctness**, **reliability**, and **early bug detection**.
 - Performed using testing frameworks like **JUnit**.
-

2. What is JUnit?

JUnit is an open-source testing framework for Java.

- Used for writing and running **unit tests**.
 - Encourages a **test-first** approach (Test-Driven Development).
 - Current version: **JUnit 5** (a.k.a. Jupiter).
-

3. JUnit Architecture (JUnit 5)

JUnit 5 is divided into three main parts:

1. **JUnit Platform** – Base for launching tests
 2. **JUnit Jupiter** – Provides new annotations and test engine
 3. **JUnit Vintage** – Supports older JUnit 3 and 4 tests
-

4. Setting Up JUnit (JUnit 5)

Using Maven(Build tool):

```
<dependency>
  <groupId>org.junit.jupiter</groupId>
  <artifactId>junit-jupiter</artifactId>
  <version>5.10.0</version>
  <scope>test</scope>
</dependency>
```

Using Gradle(Build tool):

```
testImplementation 'org.junit.jupiter:junit-jupiter:5.10.0'
```

5. JUnit Annotations (JUnit 5)

Annotation	Purpose
@Test	Marks a method as a test method
@BeforeEach	Runs before each test
@AfterEach	Runs after each test
@BeforeAll	Runs once before all tests (static method)
@AfterAll	Runs once after all tests (static method)
@Disabled	Skips the test method
@DisplayName	Custom test name
@Nested	Defines nested test classes

6. Writing a Simple Test Case

Class to be tested:

```
public class Calculator {  
    public int add(int a, int b) {  
        return a + b;  
    }  
}
```

Test Class:

```
import org.junit.jupiter.api.Test;  
import static org.junit.jupiter.api.Assertions.*;  
public class CalculatorTest {  
    @Test  
    public void testAddition() {  
        Calculator calc = new Calculator();  
        assertEquals(5, calc.add(2, 3));  
    }  
}
```

7. Common Assertions

JUnit provides several **static methods** from `org.junit.jupiter.api.Assertions`:

Method	Description
<code>assertEquals(a, b)</code>	Checks if <code>a == b</code>
<code>assertNotEquals(a, b)</code>	Checks if <code>a != b</code>
<code>assertTrue(condition)</code>	Asserts that condition is true
<code>assertFalse(condition)</code>	Asserts that condition is false
<code>assertNull(value)</code>	Asserts that value is null
<code>assertNotNull(value)</code>	Asserts that value is not null
<code>assertThrows()</code>	Asserts that a block throws exception
<code>assertAll()</code>	Groups multiple assertions together
<code>fail()</code>	Fails a test unconditionally

8. Testing Exceptions

```
@Test
void testException() {
    assertThrows(ArithmeticException.class, () -> {
        int x = 10 / 0;
    });
}
```

9. Testing with @BeforeEach and @AfterEach

```
public class SampleTest {
    Calculator calculator;

    @BeforeEach
    void setup() {
        calculator = new Calculator();
        System.out.println("Setup");
    }

    @AfterEach
    void teardown() {
        System.out.println("Teardown");
    }

    @Test
    void testAdd() {
        assertEquals(4, calculator.add(2, 2));
    }
}
```

10. Grouping Assertions with assertAll()

```
@Test
void testMultipleAssertions() {
    Calculator c = new Calculator();
    assertAll("Calculator Tests",
        () -> assertEquals(4, c.add(2, 2)),
        () -> assertNotEquals(5, c.add(2, 2))
    );
}
```

Multithreading

1. Introduction to Multithreading

Multithreading is a programming feature that allows concurrent execution of two or more parts (threads) of a program for maximum CPU utilization.

- A **thread** is a lightweight subprocess.
 - Multiple threads within a process share memory and resources.
 - Java supports multithreading through the `java.lang.Thread` class and the `Runnable` interface.
-

2. Thread in Java

What is a Thread?

A thread is an independent path of execution within a program. All threads in Java share the same address space but have their own execution stack.

Key Classes:

- `Thread` class
- `Runnable` interface

3. Creating Threads in Java

Method 1: Extending the Thread Class

```
class MyThread extends Thread {  
    public void run() {  
        System.out.println("Thread is running");  
    }  
}  
  
public class Test {  
    public static void main(String[] args) {  
        MyThread t = new MyThread();  
        t.start(); // Starts a new thread  
    }  
}
```

Notes:

- Override the `run()` method.
 - Call `start()` to begin thread execution.
-

Method 2: Implementing Runnable Interface

```
class MyRunnable implements Runnable {  
    public void run() {  
        System.out.println("Runnable thread is running");  
    }  
}  
  
public class Test {  
    public static void main(String[] args) {  
        Thread t = new Thread(new MyRunnable());  
        t.start();  
    }  
}
```

Notes:

- More flexible (supports multiple inheritance).
 - Recommended for large applications.
-

4. Runnable vs Thread

Feature	Thread Class	Runnable Interface
Inheritance	Extends Thread	Implements Runnable
Design Flexibility	Single inheritance limitation	Can extend another class
Reusability	Less reusable	More reusable
Object Sharing	Difficult to share thread instance	Runnable can be shared

Key Methods

Method	Purpose
<code>start()</code>	Starts the thread execution
<code>run()</code>	Contains code to be executed by thread
<code>sleep(ms)</code>	Pause current thread
<code>join()</code>	Waits for another thread to finish
<code>isAlive()</code>	Checks if thread is alive
<code>yield()</code>	Hint to scheduler to pause current thread

5. Multiple Threads in Java

```
class MyTask extends Thread {  
    public void run() {  
        for (int i = 1; i <= 5; i++) {  
            System.out.println("Running: " + i);  
        }  
    }  
}
```

```
public class MultiThreadDemo {  
    public static void main(String[] args) {  
        MyTask t1 = new MyTask();  
        MyTask t2 = new MyTask();  
        t1.start();  
        t2.start();  
    }  
}
```

- Output order is unpredictable due to thread scheduling.
-

6. Why Thread and Mutation Together is Dangerous

Mutating shared data from multiple threads without synchronization causes **inconsistency** and **data corruption**.

Problems:

- One thread may read stale or partially-updated data.
- Two threads may update the same variable simultaneously.

Example:

```
class Counter {  
    int count = 0;  
  
    void increment() {  
        count++;  
    }  
}
```

If multiple threads call `increment()` without synchronization, the final `count` may be incorrect.

7. Thread Safety in Java

Thread safety ensures that shared data is accessed or modified by only one thread at a time.

Ways to Achieve Thread Safety:

1. Synchronization:

```
synchronized void increment() {  
    count++;  
}
```

2. Synchronized block:

```
void increment() {  
    synchronized (this) {  
        count++;  
    }  
}
```

3. **Volatile keyword** (for visibility, not atomicity):

```
volatile boolean flag = true;
```

4. **Atomic classes** (from `java.util.concurrent.atomic`):

```
AtomicInteger count = new AtomicInteger(0);  
count.incrementAndGet();
```

5. **Thread-safe collections:**

- `ConcurrentHashMap`
- `CopyOnWriteArrayList`
- `BlockingQueue`

6. **Using locks:**

```
ReentrantLock lock = new ReentrantLock();  
lock.lock();  
try {  
    // critical section  
} finally {  
    lock.unlock();  
}
```

8. Race Condition in Java

A **race condition** occurs when multiple threads access and modify shared data concurrently, and the final outcome depends on the thread scheduling order.

Example:

```
class Counter {  
    int count = 0;  
  
    void increment() {  
        count++;  
    }  
}
```


If two threads call `increment()` at the same time, both might read the same value of `count`, increment it, and write back the same value, causing data loss.

Prevention:

- Use `synchronized`, locks, or atomic classes.

9. Thread Lifecycle / States

A thread in Java can be in one of the following states (from `Thread.State` enum):

State	Description
NEW	Thread is created but not yet started
RUNNABLE	Thread is ready to run or running
BLOCKED	Waiting to acquire a lock
WAITING	Waiting indefinitely for another thread to perform a task
TIMED_WAITING	Waiting for a specified time
TERMINATED	Thread has finished execution

State Transitions:

- `NEW` → `start()` → `RUNNABLE`
 - `RUNNABLE` → waiting/blocked → `WAITING` / `BLOCKED`
 - After task completion → `TERMINATED`
-

10. Common Thread Class Methods

Method	Description
<code>start()</code>	Starts the thread
<code>run()</code>	Code executed by the thread
<code>sleep(ms)</code>	Pauses thread for given milliseconds
<code>join()</code>	Waits for thread to die
<code>interrupt()</code>	Interrupts a sleeping or waiting thread
<code>isAlive()</code>	Checks if thread is still running
<code>yield()</code>	Hints to pause and allow other threads

11. Summary and Best Practices

- Prefer `Runnable` over extending `Thread`.
- Avoid using shared mutable state unless synchronized.
- Use thread-safe collections and atomic classes.
- Always use `start()` to begin a thread, not `run()` directly.
- Avoid using `Thread.sleep()` for synchronization purposes.
- Use modern concurrency utilities (`Executors`, `Locks`, `Atomic`, etc.).

Collection Framework

What is the Java Collection Framework?

The **Java Collection Framework (JCF)** is a unified architecture for storing and manipulating groups of objects. It includes:

- **Interfaces:** Collection, List, Set, Queue, Map, etc.
- **Implementations:** ArrayList, HashSet, LinkedHashSet, TreeSet, HashMap, LinkedHashMap, TreeMap, etc.
- **Algorithms:** Sorting, searching, shuffling via **Collections** utility class.
- **Utilities:** Comparator, Comparable, Iterator, Enumeration

Core Interfaces of the Collection Framework

Interface	Description
Collection	Root interface representing a group of objects
List	Ordered collection, allows duplicates
Set	Unordered collection, no duplicates
Queue	Elements processed in FIFO order
Map	Key-value pairs; not a part of Collection interface

1. ArrayList

Overview:

- A **resizable array** implementation of **List**.
- **Maintains insertion order.**
- **Allows duplicates**, and **null** values.
- Implements **RandomAccess**, so it supports fast access via index.

Internal Working:

- Backed by an **array** of `Object[]`.
- Initial capacity is **10** by default.
- When array is full:
 - New array is created with **1.5x old capacity**.
 - Old elements are copied to new array.
- Methods like `add()`, `get(int index)`, `remove(int index)`.

Example:

```
List<String> list = new ArrayList<>();  
list.add("Apple");  
list.add("Banana");  
list.add("Apple"); // Duplicates allowed  
System.out.println(list); // [Apple, Banana, Apple]
```

2. Set

Overview:

- A **collection that does not allow duplicates**.
- Implementations:
 - `HashSet` (unordered)
 - `LinkedHashSet` (insertion order)
 - `TreeSet` (sorted order, uses Red-Black Tree)

Internal Working:

HashSet:

- Internally uses a **HashMap** with elements as **keys** and a constant dummy value.
- Uses `hashCode()` and `equals()` methods to avoid duplicates.
- `add(E e)` → puts key into internal HashMap.

```
Set<String> set = new HashSet<>();  
set.add("One");  
set.add("Two");  
set.add("One"); // Duplicate - ignored  
System.out.println(set); // [One, Two]
```

TreeSet:

- Uses a **self-balancing Red-Black Tree**.
 - Elements must be **Comparable** or a **Comparator** must be provided.
-

3. Map

Overview:

- A **key-value** pair data structure.
- Key must be **unique**, value can be **duplicate**.
- Not part of **Collection** interface.
- Implementations:
 - **HashMap**
 - **LinkedHashMap**
 - **TreeMap**
 - **Hashtable**

Internal Working of **HashMap**

Key Concepts:

- **Key**: Unique identifier to access value
- **Value**: Data associated with a key
- **Entry**: A key-value pair is stored as an **Entry<K, V>** object
- **Bucket**: Location in an internal array (based on hash)
- **Hash Function**: **hashCode()** is used to determine index (bucket)

Flow:

1. Key's **hashCode()** is calculated.
2. **index = hash % capacity** gives bucket.
3. If bucket empty → insert entry.
4. If bucket already has entry → check with **equals()** for same key.
5. If same → update value.
6. If not same → use **Linked List or Tree (TreeNode)** to resolve collision.

Load Factor:

- Default = **0.75**
- If size exceeds **capacity * loadFactor**, resize happens (capacity doubles).

Example:

```
Map<Integer, String> map = new HashMap<>();
map.put(1, "A");
map.put(2, "B");
map.put(1, "C"); // Value updated
System.out.println(map); // {1=C, 2=B}
```

4. Comparable vs Comparator

1. Comparable Interface

- It is used to define the **default/natural sorting order** of objects.
- The class itself **implements the Comparable interface** and **overrides the compareTo()** method.

Key Points:

- Found in `java.lang` package.
- Method to implement: `public int compareTo(T o)`
- You can only have **one sorting logic** using Comparable.

Syntax Example:

```
class Student implements Comparable<Student> {
    int id;
    String name;

    Student(int id, String name) {
        this.id = id;
        this.name = name;
    }

    @Override
    public int compareTo(Student s) {
        return this.id - s.id; // Sort by id (ascending)
    }
}
```

Sorting Example:

```
List<Student> list = new ArrayList<>();
list.add(new Student(2, "B"));
list.add(new Student(1, "A"));
Collections.sort(list); // Uses compareTo() internally
```

Comparator Interface

What is it?

- It is used to define **custom sorting logic**.
- You **pass it as an argument** to `Collections.sort()` or `Arrays.sort()`.

Key Points:

- Found in `java.util` package.
- Method to implement: `public int compare(T o1, T o2)`
- Allows **multiple sorting strategies** without changing the original class.

Syntax Example:

```
class Student {
    int id;
    String name;

    Student(int id, String name) {
        this.id = id;
        this.name = name;
    }
}
```

Sorting by name:

```
Comparator<Student> byName = (s1, s2) -> s1.name.compareTo(s2.name);
```

```
List<Student> list = new ArrayList<>();
list.add(new Student(2, "B"));
list.add(new Student(1, "A"));
```

```
Collections.sort(list, byName); // Uses Comparator logic
```

Comparable vs Comparator:

Feature	Comparable	Comparator
Package	<code>java.lang</code>	<code>java.util</code>
Method	<code>compareTo(Object o)</code>	<code>compare(Object o1, Object o2)</code>
Sorting Logic	In the class itself	External to the class
Used For	Natural ordering	Custom ordering
Modifies Class?	Yes	No
Example	<code>String</code> , <code>Integer</code> , etc.	<code>Collections.sort(list, comparator)</code>

Summary Table

Collection Type	Allows Duplicates	Maintains Order	Key-Value ?	Internal Structure
ArrayList	✔ Yes	✔ Yes (index)	✗ No	Array
HashSet	✗ No	✗ No	✗ No	HashMap (keys)
LinkedHashSet	✗ No	✔ Yes	✗ No	HashMap + LinkedList
TreeSet	✗ No	✔ Sorted	✗ No	Red-Black Tree
HashMap	✗ Keys only	✗ No	✔ Yes	Hashing
LinkedHashMap	✗ Keys only	✔ Yes	✔ Yes	Hashing + LinkedList
TreeMap	✗ Keys only	✔ Sorted	✔ Yes	Red-Black Tree

Stream API

1. What is a Stream?

A **Stream** in Java (introduced in Java 8) is a **sequence of elements** supporting sequential and parallel **aggregate operations**.

Key Characteristics:

- **Does not store data** – works on source like a collection, array, or I/O channel.
 - **Lazily evaluated** – intermediate operations are not executed until a terminal operation.
 - **Can be chained** – intermediate operations return a stream.
 - **Can be parallelized** – for multi-core processing.
-

2. Stream Creation

// From Collection

```
Stream<String> stream = list.stream();
```

// From Arrays

```
Stream<Integer> intStream = Arrays.stream(new Integer[]{1, 2, 3});
```

// From values directly

```
Stream<String> s = Stream.of("a", "b", "c");
```

3. Stream Operations



Two types:

1. **Intermediate Operations** (return Stream): `map()`, `filter()`, `sorted()`, `distinct()`
 2. **Terminal Operations** (end processing): `collect()`, `forEach()`, `reduce()`, `count()`, `anyMatch()`
-

4. Key Functional Interfaces

Interface	Method Signature	Used With	Purpose
Predicate<T>	boolean test(T t)	filter()	Conditional testing
Function<T,R>	R apply(T t)	map()	Transform one type to another
Consumer<T>	void accept(T t)	forEach()	Consumes a value (e.g., prints it)
BinaryOperator<T>	T apply(T t1, T t2)	reduce()	Combines two values

5. filter() – Filtering Elements

Purpose:

Returns a stream with elements that pass the condition (true in the `Predicate<T>`).

Internal: Uses `Predicate<T>` (`boolean test(T)`)

Example:

```
List<Integer> nums = Arrays.asList(1, 2, 3, 4, 5);

List<Integer> evens = nums.stream()
    .filter(n -> n % 2 == 0) // Predicate: n -> n % 2 == 0
    .collect(Collectors.toList());

System.out.println(evens); // [2, 4]
```

Step-by-Step Breakdown:

1. What is `filter`?

```
Stream<T> filter(Predicate<? super T> predicate)
```

It keeps only elements that satisfy the condition in the predicate.

2. Using an Explicit `Predicate` Object

```
import java.util.function.Predicate;
import java.util.List;
import java.util.stream.Collectors;

public class FilterExample {
    public static void main(String[] args) {
        List<String> names = List.of("Alice", "Bob", "Charlie", "David");

        // Explicit Predicate object
        Predicate<String> startsWithA = new Predicate<>() {
            @Override
            public boolean test(String s) {
                return s.startsWith("A");
            }
        };

        // Use it in filter
        List<String> filtered = names.stream()
                                    .filter(startsWithA)
                                    .collect(Collectors.toList());

        System.out.println(filtered); // Output: [Alice]
    }
}
```

3. Convert Predicate Object to Lambda Expression

This is the lambda version of the above:

```
List<String> filtered = names.stream()
    .filter(s -> s.startsWith("A"))
    .collect(Collectors.toList());
```

6. `map()` – Transforming Elements

Purpose:

Applies a **function** to each element and returns a stream of the **resulting values**.

Internal: Uses **`Function<T, R>`** (**`R apply(T)`**)

Example:

```
List<String> names = Arrays.asList("alice", "bob");
```

```
List<String> upper = names.stream()
    .map(String::toUpperCase) // Function: s -> s.toUpperCase()
    .collect(Collectors.toList());
```

```
System.out.println(upper); // [ALICE, BOB]
```

Step-by-Step Breakdown:

1. Using an Explicit **`Function`** Object

```
import java.util.function.Function;
import java.util.List;
import java.util.stream.Collectors;

public class MapExample {
    public static void main(String[] args) {
        List<String> names = List.of("alice", "bob", "charlie");

        // Explicit Function object
        Function<String, String> toUpperCase = new Function<>() {
            @Override
            public String apply(String s) {
                return s.toUpperCase();
            }
        };

        List<String> upperNames = names.stream()
            .map(toUpperCase)
            .collect(Collectors.toList());

        System.out.println(upperNames); // Output: [ALICE, BOB, CHARLIE]
    }
}
```

2. Convert to Lambda Expression

Same logic, written with a lambda:

```
List<String> upperNames = names.stream()
    .map(s -> s.toUpperCase())
    .collect(Collectors.toList());
```

7. **reduce()** – Aggregating Elements

Purpose:

Performs a **reduction** (accumulation) operation on stream elements.

Internal: Uses **BinaryOperator<T> (T apply(T, T))**

Example 1: Sum

```
int sum = nums.stream()
    .reduce(0, (a, b) -> a + b); // identity + BinaryOperator

System.out.println(sum); // 15
```

Example 2: Product without identity

```
Optional<Integer> product = nums.stream()
    .reduce((a, b) -> a * b);

product.ifPresent(System.out::println);
```

8. **collect()** – Terminal Collector

Purpose:

Collects stream results into a **collection**, **string**, or **summary**.

Examples:

```
List<String> list = stream.collect(Collectors.toList());  
Set<Integer> set = stream.collect(Collectors.toSet());  
String result = stream.collect(Collectors.joining(", "));
```

9. **forEach()** – Consuming Elements

Purpose:

Performs an action on each element (like printing).

Internal: Uses **Consumer<T>** (**void accept(T)**)

Example:

```
names.stream().forEach(name -> System.out.println(name));
```

10. Other Useful Stream Methods

Method	Description
distinct()	Removes duplicate elements
sorted()	Sorts elements naturally or with comparator
limit(n)	Limits the result to n elements
skip(n)	Skips first n elements
count()	Returns count of elements (terminal op)
anyMatch()	Returns true if any element matches
allMatch()	True if all elements match predicate
noneMatch()	True if no elements match predicate

11. Working with Custom Objects

Example:

```
class Employee {  
    String name;  
    int salary;  
    // constructor, getters  
}
```

```
List<Employee> employees = ...;
```

```
List<String> highEarners = employees.stream()  
    .filter(emp -> emp.getSalary() > 50000)  
    .map(Employee::getName)  
    .collect(Collectors.toList());
```

12. Parallel Streams

Use to leverage multi-core CPUs:

```
list.parallelStream()  
    .filter(...)  
    .map(...)  
    .collect(...);
```

Use carefully: only when thread-safety and performance justify it.

13. Stream Pipeline Anatomy

```
source.stream()           // Creation  
    .filter(...)          // Intermediate op  
    .map(...)              // Intermediate op  
    .sorted(...)           // Intermediate op  
    .collect(Collectors.toList()); // Terminal op
```

- Only when **terminal** op is called, pipeline executes.
-

14. Stream vs Collection

Stream	Collection
Doesn't store data	Stores data
Lazy evaluation	Eager evaluation
Can be infinite	Always finite
Functional-style	Object-oriented style
No modification of original data	Can modify data

15. Real-World Example

```
List<String> names = Arrays.asList("John", "", "Jane", " ", "Jack");
```

```
int totalChars = names.stream()
    .map(String::trim)
    .filter(s -> !s.isEmpty())
    .map(String::length)
    .reduce(0, Integer::sum);
```

```
System.out.println(totalChars); // 12
```

Summary Diagram/Flow

```
Collection → Stream → [filter] → [map] → [sorted] → [collect/reduce/forEach]
```


JDBC and Database connectivity

What is JDBC?

JDBC is a Java API that allows you to connect and interact with relational databases like MySQL, Oracle, PostgreSQL, etc.

It lets you:

- Connect to a database
 - Execute SQL queries (SELECT, INSERT, UPDATE, DELETE)
 - Retrieve results
 - Manage transactions
-

JDBC Architecture

JDBC involves:

1. Driver Manager
 2. Driver
 3. Connection
 4. Statement
 5. ResultSet
-

JDBC Workflow (Steps)

1. Import JDBC classes
 2. Load and register JDBC driver
 3. Establish connection
 4. Create a statement
 5. Execute SQL query
 6. Process the result
 7. Close the connection
-

How to Create JDBC Connection

Requirements

- A database (e.g., MySQL)
- JDBC driver (e.g., [mysql-connector-java.jar](#))
- Proper URL, username, and password

Key JDBC Classes

Class/Interface	Purpose
DriverManager	Manages JDBC drivers
Connection	Connects to DB
Statement	Executes static SQL
PreparedStatement	Executes parameterized SQL
ResultSet	Holds data returned from SELECT

JDBC URL Format

[jdbc:mysql://host:port/database](#)

Example:

[jdbc:mysql://localhost:3306/mydatabase](#)

Tips

- Always close **Connection**, **Statement**, and **ResultSet** in **finally** block (or use try-with-resources).
- Prefer **PreparedStatement** for security (prevents SQL injection).
- Use proper JDBC driver version.

```
Demo.java x
1 import java.sql.*; // Importing all classes from the java.sql package for database operations
2 public class Demo {
3     public static void main(String[] args) throws SQLException {
4         // SQL query to select all columns from the Student table where the id is 20
5         String sql = "select * from Student where id = 20";
6
7         // JDBC URL to connect to the MySQL database named SchoolDb on localhost and port 3306
8         String url = "jdbc:mysql://localhost:3306/SchoolDb";
9         // Username for the database
10        String username = "root";
11        // Password for the database
12        String password = "admin";
13        // Establishing a connection to the database using the given URL, username, and password
14        Connection con = DriverManager.getConnection(url, username, password);
15
16        // Creating a Statement object to send SQL queries to the database
17        Statement st = con.createStatement();
18
19        // Executing the SQL query and storing the result in a ResultSet
20        ResultSet resultSet = st.executeQuery(sql);
21        // Moving the cursor to the first row of the result set
22        resultSet.next();
23        // Retrieving values from the result set using column indices
24        String id = resultSet.getString(columnIndex: 1); // Getting the value of the 1st column (id)
25        String name = resultSet.getString(columnIndex: 2); // Getting the value of the 2nd column (name)
26        String age = resultSet.getString(columnIndex: 3); // Getting the value of the 3rd column (age)
27        String grade = resultSet.getString(columnIndex: 4); // Getting the value of the 4th column (grade)
28        String email = resultSet.getString(columnIndex: 5); // Getting the value of the 5th column (email)
29        // Printing the retrieved values to the console
30        System.out.println(id + " " + name + " " + age + " " + grade + " " + email);
31        // Closing the connection to free database resources
32        con.close();
33    }
34 }
```

JDBC Student CRUD Project

Setup

Database Configuration

```
static final String URL = "jdbc:mysql://localhost:3306/SchoolDB";
static final String USER = "root";
static final String PASSWORD = "admin";
```

- **URL:** JDBC connection string to connect to **SchoolDB** on MySQL at **localhost:3306**.
 - **USER/PASSWORD:** Credentials to authenticate with the MySQL server.
-

Establishing Connection

```
static Connection getConnection() throws SQLException {
    return DriverManager.getConnection(URL, USER, PASSWORD);
}
```

- Uses **DriverManager.getConnection()** to return a **Connection** object.
- Every method in CRUD calls this method to get a live connection to the DB.

CRUD OPERATIONS

1. CREATE – Insert a New Student

```
// 1. CREATE - Insert a new student
static void createStudent(String name, int age, String grade, String email) throws SQLException {
    String sql = "INSERT INTO Student (name, age, grade, email) VALUES (?, ?, ?, ?)";
    try (Connection con = getConnection(); PreparedStatement pst = con.prepareStatement(sql)) {
        pst.setString(parameterIndex: 1, name);
        pst.setInt(parameterIndex: 2, age);
        pst.setString(parameterIndex: 3, grade);
        pst.setString(parameterIndex: 4, email);
        pst.executeUpdate();
        System.out.println("Student inserted successfully.");
    }
}
```

What's Happening:

- SQL insert query with placeholders (?).
- `PreparedStatement` is used to safely insert user data.
- Values are passed using `setXXX()` methods.
- `executeUpdate()` runs the insert.
- Message is printed upon success.

2. READ – Get a Student by ID

```
// 2. READ - Get a student by ID
static void readStudent(int id) throws SQLException { 1 usage
    String sql = "SELECT * FROM Student WHERE id = ?";
    try (Connection con = getConnection(); PreparedStatement pst = con.prepareStatement(sql)) {
        pst.setInt( parameterIndex: 1, id);
        ResultSet rs = pst.executeQuery();
        if (rs.next()) {
            System.out.println("ID: " + rs.getInt( columnLabel: "id"));
            System.out.println("Name: " + rs.getString( columnLabel: "name"));
            System.out.println("Age: " + rs.getInt( columnLabel: "age"));
            System.out.println("Grade: " + rs.getString( columnLabel: "grade"));
            System.out.println("Email: " + rs.getString( columnLabel: "email"));
        } else {
            System.out.println("Student not found.");
        }
    }
}
```

What's Happening:

- Prepares a SQL SELECT query with a placeholder for `id`.
- `ResultSet` is used to fetch the returned record.
- If `rs.next()` is true → row exists, so print its data.
- If no record → prints "Student not found".

3. UPDATE – Update Student's Name by ID

```
// 3. UPDATE - Update student name by ID
static void updateStudentName(int id, String newName) throws SQLException { 1 usage
    String sql = "UPDATE Student SET name = ? WHERE id = ?";
    try (Connection con = getConnection(); PreparedStatement pst = con.prepareStatement(sql)) {
        pst.setString( parameterIndex: 1, newName);
        pst.setInt( parameterIndex: 2, id);
        int rows = pst.executeUpdate();
        System.out.println(rows + " row(s) updated.");
    }
}
```

What's Happening:

- Prepares an SQL **UPDATE** statement to change name for the given ID.
- Executes **executeUpdate()** and returns the number of affected rows.
- Prints how many rows were updated (0 if not found).

4. DELETE – Remove a Student by ID

```
// 4. DELETE - Delete student by ID
static void deleteStudent(int id) throws SQLException { 1 usage
    String sql = "DELETE FROM Student WHERE id = ?";
    try (Connection con = getConnection(); PreparedStatement pst = con.prepareStatement(sql)) {
        pst.setInt( parameterIndex: 1, id);
        int rows = pst.executeUpdate();
        System.out.println(rows + " row(s) deleted.");
    }
}
```

What's Happening:

- Uses SQL **DELETE** query where **id** matches.
 - **executeUpdate()** deletes the record.
 - Prints how many rows were deleted.
-

Main Menu – CLI Interface

```
public static void main(String[] args) throws SQLException {
    Scanner sc = new Scanner(System.in);
    while (true) {
        System.out.println("\n=== Student CRUD Menu ===");
        System.out.println("1. Add Student");
        System.out.println("2. Get Student by ID");
        System.out.println("3. Update Student Name");
        System.out.println("4. Delete Student");
        System.out.println("5. Exit");
        System.out.print("Enter your choice: ");
        int choice = sc.nextInt();

        switch (choice) {
            case 1 -> {
                sc.nextLine(); // consume newline
                System.out.print("Name: ");
```

```
String name = sc.nextLine();
System.out.print("Age: ");
int age = sc.nextInt();
sc.nextLine();
System.out.print("Grade: ");
String grade = sc.nextLine();
System.out.print("Email: ");
String email = sc.nextLine();
createStudent(name, age, grade, email);
}
case 2 -> {
    System.out.print("Enter ID: ");
    int id = sc.nextInt();
    readStudent(id);
}
case 3 -> {
    System.out.print("Enter ID: ");
    int id = sc.nextInt();
    sc.nextLine();
    System.out.print("Enter new name: ");
    String name = sc.nextLine();
    updateStudentName(id, name);
}
case 4 -> {
    System.out.print("Enter ID: ");
    int id = sc.nextInt();
    deleteStudent(id);
}
case 5 -> {
    System.out.println("Exiting...");
    return;
}
default -> System.out.println("Invalid choice.");
}
}
```